# 《地下水动力学》实验指导书

杨国勇

水文与水资源系

资源与地球科学学院

中国矿业大学

2026 年

# 目录


**序言** **1**

**搭建自己的 Python 编程环境** **3**

创建 Python 虚拟环境 . . . 3

实验所用库的简介 . . . 6

生成可交互的用 Jupyter Notebook 文档 . . . 6

Jupyter 服务的启动方式 . . . 6

Notebook 的 cell 类型及操作 . . . 7

在 Code 单元中使用 matplotlib 绘图 . . . 7

NoteBook 代码运行顺序和变量作用域 . . . 7

Markdown 语法 . . . 7

求参辅助函数 . . . 10

**实验 1 Dupuit 公式与影响半径经验公式求参** **13**

**实验 2 交互式配线法求参** **19**

Theis 公式的配线法 . . . 19

Hantush - Jacob 公式的配线法 . . . 25

**实验 3 交互式拐点法求参** **33**

**实验 4 交互式直线图解法求参** **39**

**实验 5 最小二乘法应用** **43**

最小二乘法原理 . . . 43

确定 Q - s 曲线类型 . . . 45

Jacob 公式线性拟合 . . . 50

Theis 公式非线性拟合 . . . 56

**实验 6 一维显式有限差分法** **61**

有限差分基础知识 . . . 61

显示差分与隐式差分格式 . . . 61

显式差分格式 . . . 62

隐式差分格式 . . . . . 62
**Neumann** 边界条件 . . . . . 63

**附录井函数计算** **69**
Theis 井函数 . . . . . 69
Hantush-Jacob 井函数 . . . . . 70

# 序言

地下水动力学是研究地下水在多孔介质与裂隙介质中运动规律的科学。该学科综合运用理论解析、物理模拟与数值模拟三种方法，为地下水资源评价、矿井涌水预测、污染物迁移模拟等实际问题提供理论依据。

依据抽水试验数据反演含水层参数（如导水系数、储水系数、越流系数等），是连接理论与实际应用的重要纽带。地下水动力学实验不仅是对课堂理论的验证，更是对真实水文地质勘察工作流程的模拟。

长期以来，配线法、直线图解法、拐点法等手工图解法作为求参的主要手段，但其操作繁琐、主观性强，且难以处理大规模数据。随着计算技术与编程工具的普及，地下水动力学实验教学正逐步从“手工图解法”向“编程互动式分析”转型。编写本实验指导书正是为了顺应这一教学发展趋势。

实验共设计七个循序渐进的项目，内容可分为四个相互关联的环节：

第一环节为编程环境搭建，主要介绍 Python 编程环境的部署方法、配套工具与第三方程序库的使用。本环节是后续所有实验开展的技术基础，要求学习者优先完成环境配置工作。

第二环节是通过编程实现经典方法，即将 Dupuit 公式、影响半径经验公式求参、配线法、拐点法、直线图解法等传统人工图解法转化为程序实现。编写代码的过程有助于深入理解这些方法的基本原理。

第三环节应用最小二乘原理，内容包括确定稳定流抽水实验的 Q-s 曲线类型、计算井损系数与井损，以及基于最小二乘原理的求参方法。此部分涉及一定的优化计算。

第四环节为一维显式差分数值方法应用。相较于解析方法，数值模拟技术能够有效应对复杂的水文地质条件。本环节旨在从“简单解析条件”拓展至“复杂实际场景”，并对前述解析法计算结果进行验证，提高学生的学习兴趣与技能。

实验内容未涉及高深数学理论或复杂编程技巧，所介绍均为初学者可理解的基础知识。期望学生通过“编写代码解决水文地质问题”这一方式，不仅掌握经典模型的核心思想与求参技术，更能逐步建立运用现代计算工具分析地下水运动的能力。

本指导书的目标并非替代理论推导，而是使抽象的水文地质问题变得可操作、可调整、可验证。旨在让地下水动力学的学习回归“发现问题—建立模型—求解验证—编程实现”的科学探索过程，同时为学生未来在科研或工程实践中使用 Python 奠定基础。

由于编者水平与实践经验有限，书中难免存在疏漏、不足乃至错误之处。恳请使用本指导书

的教师、同学及工程技术人员，在实际使用过程中提出宝贵意见，以便后续修订时不断完善。

编 者

2026 年 6 月 20 日

# 搭建自己的 Python 编程环境

Python 是一种面向对象、解释型编程语言，其语法简洁，学习门槛较低。该语言具备丰富且功能强大的标准库与第三方库，常被誉为“胶水语言”，能够便捷地集成其他语言编写的模块。在科学计算领域，Python 拥有成熟完善的软件包生态，例如 NumPy、SciPy、Matplotlib 等，为该领域的应用提供了有力支持。

在搭建 Python 编程环境时，可采用两种主要方式：一是选择集成度较高的发行包（如 Anaconda），二是基于 Python 官方发行版进行自主配置。

Anaconda 是一款常用的 Python 发行包，集成了众多科学计算与数据分析所需的库，包括前述的 NumPy、SciPy、Matplotlib 以及 Pandas 等。其内置的 conda 包管理工具，支持通过简洁的命令行操作创建独立的 Python 环境，并可轻松完成软件包的安装、更新与卸载，从而大幅简化环境部署流程。

Miniconda 是 Anaconda 的轻量级版本，仅包含 Python 解释器与 conda 包管理器，未预装其他库。其安装包体积较小，适用于对存储空间较为敏感或希望根据实际需求灵活安装库的用户。

针对《地下水动力学》课程实验的编程需求，可根据具体情况选择 Anaconda 或 Miniconda 等发行包，亦可基于 Python 官方版本进行手动环境配置。

本节内容将重点阐述基于 Python 官方发行版搭建编程环境的方法。

## 创建 Python 虚拟环境

**安装 Python 发行版**

用户应根据所用操作系统版本，前往 https://www.python.org/downloads 下载对应的 Python 发行版，并可同时下载 Python 帮助文档。

以 Windows 操作系统为例，Python 安装程序提供了两种安装方式。其中，`Install Now` 为默认安装模式，其安装路径与组件均采用预设配置。`Customize installation` 则为自定义安装模式，用户可在此过程中根据需求增减功能模块，并指定安装路径。

安装程序还提供了两个可选配置项：`Use admin privileges when installing py.exe` 与 `Add python.exe to PATH`。前者表示使用管理员权限安装 `py.exe`，后者则会将 Python 可执行文件添加至系统环境变量。建议用户同时勾选此两项，以便其他非管理员账户无需重复安装，且系统可自动识别已安装的 Python。

**在指定文件夹中创建 Python 虚拟环境**

通过 `Win + R` 组合键并输入 `cmd` 打开 Windows 命令提示符，随后切换至目标目录。例如，若计划在 D 盘根目录下创建名为 `py_groundwater` 的虚拟环境，则需依次执行以下命令：

```
D:
python -m venv py_groundwater
```

系统将在 D 盘根目录下生成 `py_groundwater` 文件夹，其中包含构建虚拟环境所需的全部文件。

**激活虚拟环境**

要激活该虚拟环境，需运行 `scripts` 子目录中的批处理文件：

```
py_groundwater\scripts\activate.bat
```

若要退出虚拟环境，则运行：

```
py_groundwater\scripts\deactivate.bat
```

**创建进入虚拟环境的快捷方式**

退出虚拟环境后，如需再次进入，仍需重新执行 `activate.bat`。为提升操作便捷性，可在桌面创建指向命令提示符的快捷方式，并将其属性中的目标 修改为：

```
%windir%\system32\cmd.exe /K D:\py_groundwater\scripts\activate.bat
```

同时将 起始位置 设置为 `d:\`（此处假设虚拟环境位于 D 盘根目录，且用户编写的 Notebook 文件将保存在 D 盘某子目录中）。关闭终端窗口即自动退出虚拟环境；再次进入时，仅需双击该快捷方式即可。

**使用 Pip 工具在虚拟环境中安装所需库的安装**

Pip 是 Python 内置的包管理器，无需单独安装，可用于安装、更新或卸载官方库。Pip 通过访问 Python 包索引服务器 PyPI（https://pypi.org）实现版本管理。由于 PyPI 主服务器位于境外，国内用户在使用 Pip 时可能遇到网络超时问题，改用国内镜像站点可有效避免此类情况。

常用 PyPI 国内镜像站点如下：

- 清华大学镜像：
  https://pypi.tuna.tsinghua.edu.cn/simple
- 阿里云镜像：
  http://mirrors.aliyun.com/pypi/simple/
- 中国科学技术大学镜像：
  https://pypi.mirrors.ustc.edu.cn/simple/
- 华中科技大学镜像：
  http://pypi.hustunique.com/
- 山东理工大学镜像：
  http://pypi.sdutlinux.org/
- 豆瓣镜像：
  http://pypi.douban.com/simple/

Pip 常用操作指令如下：

- 查看 Pip 版本信息：
  `pip --version`

- 安装指定库：
  `pip install some-package`
- 卸载指定库：
  `pip uninstall some-package`
- 升级指定库：
  `pip install some-package --upgrade` 或 `pip install -U some-package`
- 查看指定库信息：
  `pip show some-package`
- 查看所有可升级的库：`pip list --outdated`
- 查看已安装库的版本号：`pip freeze`
- 将 `pip freeze` 输出保存到 requirements.txt 文件，其中包含当前虚拟环境中安装的所有 Python 包及其版本：
  `pip freeze > requirements.txt`
- 从 requirements.txt 文件安装所有依赖包（快速复刻相同的 Python 包配置）：
  `pip install -r requirements.txt`
- 从 PyPI 镜像安装库（以清华大学镜像为例）：
  `pip install -i https://pypi.tuna.tsinghua.edu.cn/simple some-package`
- 将 PyPI 镜像设为默认镜像（以清华大学镜像为例）：
  `pip config set global.index-url https://pypi.tuna.tsinghua.edu.cn/simple`
  若需升级的库数量较多，可使用 pip-review 工具：
- 安装 pip-review：
  `pip install pip-review`
- 使用 pip-review 交互式升级全部库：
  `pip-review --local --interactive`
  **地下水动力学课程实验用到的库可以用如下命令一起安装：**

```
pip install numpy scipy matplotlib jupyter jupyterlab ipywidgets ipympl
```

上述模块存在依赖关系，安装过程将自动处理。安装完成后，用户即可在虚拟环境中使用这些库进行科学计算、数据分析与可视化等操作。

**操作顺序总结：**

1. 安装 Python 发行版
2. 创建 Python 虚拟环境
3. 激活虚拟环境
4. 创建进入虚拟环境的快捷方式
5. 将清华大学镜像设为 PyPI 默认镜像
6. 一并安装实验所需库

## 实验所用库的简介

- IPython：
  一种交互式 Python 解释器，提供自动补全、语法高亮、历史记录等功能。它不仅是解释器，更是构建 Python 科学计算体系与实现交互可视化的平台，为科学计算工作流提供了高效、灵活的交互环境。
- NumPy：
  Python 科学计算领域的核心基础库，其核心为 N 维数组对象 ndarray，为处理多维数组提供了高效手段，是 Python 科学计算生态的基石。
- SciPy：
  一个功能丰富的开源科学计算库，集成了统计、积分、插值、优化、图像处理等常用工具模块，为科学研究与工程应用提供全面的数值计算支持。
- Matplotlib：
  专注于 2D 绘图的库，旨在提供与 MATLAB 类似的绘图接口，可生成高质量的线图、散点图、等高线图、条形图及 3D 图形等，广泛应用于数据分析与科学研究。
- IPyWidgets：
  为 Jupyter Notebook 设计的交互功能增强工具，提供滑块、按钮、下拉菜单等交互元素，可与代码单元紧密结合，实现交互式图表创建，通常随 Jupyter 默认安装。
- ipympl：
  在 Jupyter 环境中充当交互式后端，增强了 Jupyter Notebook 或 JupyterLab 中的交互式绘图功能，支持缩放、平移、选取等操作，提升了数据探索的交互体验。

## 生成可交互的用 Jupyter Notebook 文档

Jupyter Notebook 是一款基于网页的交互式计算应用，允许用户在称为 `cell`（单元）的结构中编写代码，并通过 `Shift + Enter` 执行当前单元代码，结果即时显示在代码下方。该设计实现了代码编写、执行与结果查看的无缝衔接。Jupyter Notebook 还支持多种编程语言、多媒体嵌入及 Markdown 语法，使其成为数据处理、可视化与机器学习的常用工具。其文档后缀为.ipynb，实验中的示例文档即采用此格式。JupyterLab 作为 Jupyter 项目的组成部分，构建于 Web 用户界面之上，与 Jupyter Notebook 共享同一服务器，并使用相同的笔记本文档格式（.ipynb）。

### Jupyter 服务的启动方式

在终端中输入 `jupyter notebook` 或 `jupyter-notebook` 命令可启动 Jupyter Notebook 服务，系统初始化后将在浏览器中打开用户界面，显示启动目录下的文件列表。用户可在此创建、编辑、运行 Notebook 文件。输入 `jupyter lab` 或 `jupyter-lab` 命令将启动 JupyterLab 服务，其界面提供导航栏，可视为 Jupyter Notebook 的升级版本。若浏览器未自动打开，可将终端中显示的 `http://localhost:8888/tree?token=...` 或 `http://127.0.0.1:8888/tree?token=...` 链接复制至浏览器地址栏手动打开。将链接中的 `tree` 替换为 `lab` 即可打开 JupyterLab 界面。如需使用其他路径下的 Notebook 文件，需先在终

端中使用 `cd` 命令切换至目标路径，再启动 Jupyter 服务。

### Notebook 的 cell 类型及操作

Notebook 的 `cell` 有三种格式：Code、Markdown、Raw。

- **Code 单元：**
  用户可在此编写 Python 或其他支持语言的代码，按 `Shift + Enter` 执行，结果即时显示，适用于数据读取、处理与模型训练等。
- **Markdown 单元：**
  依据 Markdown 语法书写文本内容，按 `Shift + Enter` 渲染为格式化文本，适用于添加文档注释与说明性文字。
- **Raw 单元：**
  用于直接输入和显示原始内容，不经处理或渲染，适用于展示原始数据或特定格式代码片段。

用户也可通过点击 `Run` 按钮执行单元操作。

### 在 Code 单元中使用 matplotlib 绘图

在 Code 单元中，用户可利用 Matplotlib 库进行绘图。Matplotlib 的工作机制涉及 `frontend`（前端，用户编写的绘图逻辑）与 `backend`（后端，负责图形显示的底层代码）。通过魔法命令可指定后端并启用交互功能，例如`%matplotlib ipympl` 或 `%matplotlib widget` 可为绘图提供缩放、平移等交互操作。可用 `%matplotlib --list` 查看所有相关魔法命令。

### NoteBook 代码运行顺序和变量作用域

Notebook 中代码按 Code 单元的顺序依次运行，单元前的编号即运行顺序。变量具有全局作用域：一旦某个单元被执行，其中定义的变量即可被后续单元访问和修改。此特性可能导致代码修改时产生意外错误，必要时可重启 `kernel`（内核）以重置所有变量和运行状态，然后按顺序重新运行单元代码。

## Markdown 语法

Jupyter-Notebook 或 Jupyter-lab 的帮助菜单中包含 Markdown 帮助内容，用户可自行查阅。由于 Markdown 语法简洁，此处作简要介绍。

Markdown 是一种轻量级标记语言，用于将纯文本快速转换为格式良好的文档。其主要语法规则如下：

- **标题：**
  使用 `#` 表示标题，数量代表级别，如`# 一级标题`、`## 二级标题`，最多支持六级标题。
- **段落与换行：**
  段落间以空行分隔。行内换行需在结尾添加两个空格后回车。
- **字体格式：**
  - 加粗：使用 `**` 或 `__` 包裹内容，如`**加粗文字**`。

  - 斜体：使用 * 或 _ 包裹内容，如*斜体文字*。
- **列表：**
  - 有序列表：以数字加点开头，如 1. 第一项。
  - 无序列表：以 -、* 或 + 开头，如- 列表项。
- **链接和图片：**
  - 链接：格式为 [链接文字](链接地址)，如 [百度](https://www.baidu.com)。
  - 图片:格式为 ![图片标题](图片地址),如![风景图](https://example.com/image.jpg)。
- **引用：**

  使用 > 符号表示引用内容，如：> 这是一段引用的文字。
- **数学公式**

  Markdown 借助 LaTeX 语法呈现数学公式。行内公式置于 $ 之间，如 $a + b = c$；行间公式使用 $$ 包裹，如$$\int_{a}^{b} f(x)dx$$。LaTeX 符号书写方式可参考 wikibooks 或在线编辑器 https://www.latexlive.com/。

Markdown 在技术文档撰写、博客平台、个人知识管理等领域广泛应用，因其语法简单、专注于内容，适合快速记录与分享。

**Matplotlib 中文显示配置**

matplotlib 绘图默认使用 sans-serif（无衬线字体），设置好中文字体后才能正确显式中文。matplotlib 支持 Latex 数学公式，如 $y=\sin(x)$。STIX 字体可以用衬线字体显式数学公式，比缺省设置美观。要显式含数学符号的图形标题时，加上前缀 r（raw string literal），如 label = r"$f(x)=\sin(x)$"，作用是让字符串里的反斜杠\ 保持原义，不会被当作转义字符。如果与 f-string 格式化字符串同时使用，花括号需要双写 { }，因为单括号会被 f-string 解释为占位符。如 label=rf"$n_{{\mathrm{{max}}}} = {nmax}$")。

实验绘图时显示中文汉字的配置如下：

```
import matplotlib.pyplot as plt
plt.rcParams['font.family'] = ['Times New Roman']
plt.rcParams['mathtext.fontset'] = 'stix'  # 显示数学公式（STIX 字体）
plt.rcParams['axes.unicode_minus'] = False  # 解决中文负号不能正确显示的问题
```

值得注意的是，plt.rcParams['font.family'] = ['Times New Roman'] 是避免对数刻度标注时无法显示负数上标的错误。当图形标题、数轴标签、图例标题及图例标签文本显示汉字乱码时，应该分别设置字体。

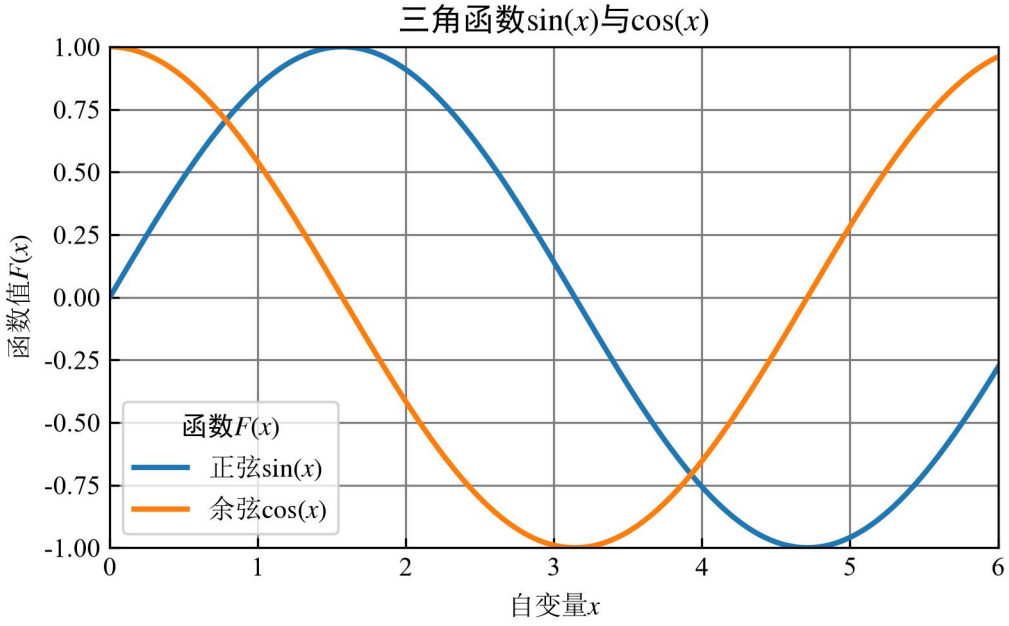
Table 1: 中文字体属性表

| 参数 | 可用接口 | 作用 | 示例 |
|---|---|---|---|
| fontfamily | title/xlabel/ylabel/text/annotate | 单独指定字体族 | ax.set_title("xx", fontfamily = "SimHei") |

| 参数 | 可用接口 | 作用 | 示例 |
|---|---|---|---|
| fontdict | title/xlabel/ylabel/text/annotate | 批量配置全套字体样式 | `fontdict = {"family": "SimHei", "size": 12}` |
| prop/fontproperties | 仅 ax.legend() | 图例专属字体配置 | `ax.legend(prop = {"family": "KaiTi"})` |

以下为示例：

```
import numpy as np
import matplotlib.pyplot as plt
plt.rcParams['mathtext.fontset'] = 'stix'
plt.rcParams['axes.unicode_minus'] = False
x = np.linspace(0, 2 * np.pi, 1000)  # 0~2，1000 个采样点
y_sin = np.sin(x)
y_cos = np.cos(x)
fig, ax = plt.subplots()
ax.plot(x, y_sin, linewidth=2, label=r"正弦$\sin(x)$")
ax.plot(x, y_cos, linewidth=2, label=r"余弦$\cos(x)$")
ax.set(xlim=(0, 6), ylim=(-1, 1))
ax.set_xlabel(r'自变量$x$', fontfamily="SimSun")
ax.set_ylabel(r'函数值$F(x)$', fontfamily="SimSun")
ax.set_title(r'三角函数$\sin(x)$与$\cos(x)$', fontfamily="SimHei")
ax.legend(
    prop={"family": "SimSun"}, title=r"函数$F(x)$",
    title_fontproperties={'family': 'SimHei'})
ax.grid(True, color="gray", linestyle="-")
plt.tight_layout()
plt.savefig(f"fig_files/字体设置示例.png",dpi=300,bbox_inches='tight')
plt.show()
```




## 求参辅助函数

为便于后续实验调用，此处定义估算初始水文地质参数的函数 `initial_parameters(.)`。该函数接收时间数组 `t`、降深数组 `s`、抽水量 `Q`、观测孔距离 `r`，从 `(t, s)` 序列中抽取两组数据按 Jacob 近似公式计算参数，并返回导水系数 `T`、储水系数 `S`、零降深截距时间 `t0` 和直线斜率 `slope`。

```
import numpy as np
def initial_parameters(t, s, Q, r):
    """
    根据 Jacob 近似公式估算导水系数 T 和储水系数 S。
    参数:
        t : array, 时间数组 (单位需与抽水量 Q 一致, 如 min)
        s : array, 降深数组 (m)
        Q : float, 抽水量 (m^3/min)
        r : float, 观测孔距离 (m)
    返回:
        T : float, 导水系数 (m^2/min)
        S : float, 储水系数 (无量纲)
        t0: float, 降深为零时的截距时间 (min)
        slope: float, s-lg(t) 直线段的斜率
    """
    # 按序列长度取中间相邻的两点
    idx = len(t) // 2 - 1
```

```
    # 也可按时间取中部相邻的两点
    # idx = np.argmin(np.abs(t / t[-1] - 0.5)) - 1
    slope = (s[idx+1] - s[idx]) / np.log10(t[idx+1] / t[idx])
    T = 0.183 * Q / slope
    t0 = t[idx] * 10**(-s[idx] / slope)
    S = 2.25 * T * t0 / r**2
    return T, S, t0, slope
```

**注意：**该函数要求时间 `t` 的单位与抽水量 `Q` 中的时间单位一致（例如均为分钟），降深 `s` 的单位为米，距离 `r` 的单位为米。

# 实验 1 Dupuit 公式与影响半径经验公式求参

**影响半径的经验公式**

- **吉哈尔特公式（承压水）**
  公式为：

$$R = 10 s_w \sqrt{K} \tag{1.1}$$

- **库萨金公式（针对潜水）**
  公式为：

$$R = 2 s_w \sqrt{K H_0} \tag{1.2}$$

在上述公式中，$K$ 表示渗透系数（单位：$m/d$）；$s_w$ 为设计降深（单位：$m$）；$H_0$ 为自底板算起的含水层静止水位厚度（单位：$m$）。

需注意，上述公式为经验公式，当变量采用其他单位时，公式中的常系数会相应变化。

**单孔抽水试验计算**

在单孔抽水试验中，可通过 Dupuit 公式与经验公式构建迭代表达式，联立求解渗透系数 $K$ 与影响半径 $R$。

- **承压水井**
  表达式为：

$$\begin{cases} K &= \dfrac{Q}{2\pi M s_w} \ln \dfrac{R}{r_w} \\ R &= 10 s_w \sqrt{K} \end{cases} \tag{1.3}$$

- **潜水井**
  表达式为：

$$\begin{cases} K &= \dfrac{Q}{\pi (2H_0 - s_w) s_w} \ln \dfrac{R}{r_w} \\ R &= 2 s_w \sqrt{K H_0} \end{cases} \tag{1.4}$$

**Python 程序实现**

建议使用 NumPy 库，并通过别名 `np` 导入：`import numpy as np`。NumPy 在数组运算方面具有较高效率，其数组索引从 0 开始。`numpy.array` 函数可将 Python 列表转换为 NumPy 数组；`numpy.ones` 与 `numpy.zeros` 可用于创建元素全为 1 或 0 的数组。常用数学函数在 NumPy 中均有对应实现，如 `numpy.sin`、`numpy.abs`、`numpy.exp`、`numpy.float_power`、`numpy.log10` 等。

`numpy.set_printoptions` 可用于设置数组的显示格式。例如，`numpy.set_printoptions( precision = 4 )` 可控制显示 4 位小数；`numpy.set_printoptions(formatter={'float': '{:.2f}'.format})` 可将浮点数统一显示为 2 位小数。

`numpy.vectorize` 可将自定义函数向量化，使该函数能够接收向量参数并返回向量结果。

以承压水井为例，定义名为 `calculate_permeability_1` 的函数，参数包括抽水量 $Q$、含水层厚度 $M$、抽水井降深 $s_w$、抽水井半径 $r_w$，以及误差限 `tolerance=1e-4`，返回值为渗透系数 $K$ 和影响半径 $R$。

计算过程为：首先设定 $K$ 的初始值，随后按经验公式计算 $R$，再根据 Dupuit 公式计算新的 $K$，如此循环迭代，直至两次计算结果的绝对误差小于设定的误差限为止。

**例 1**

某承压含水层厚度为 16.5 m。采用 3 眼完整井进行稳定流抽水试验，根据下表数据求取渗透系数 $K$ 与影响半径 $R$（计算精度要求为 $10^{-4}$）。

Table 2: 抽水试验数据

| 序号 | 抽水井半径 (m) | 抽水量 ($m^3/d$) | 抽水井降深 (m) |
|---|---|---|---|
| 1 | 0.4 | 320.54 | 1.16 |
| 2 | 0.4 | 421.63 | 1.60 |
| 3 | 0.4 | 536.54 | 1.90 |

**误差度量方式的选择**

在数值计算的迭代过程中，常用的误差度量方式包括绝对误差与相对误差。

- **绝对误差：**
  指两次迭代结果差值的绝对值。若当前迭代得到的渗透系数为 $K_{\text{new}}$，上一次迭代结果为 $K_{\text{old}}$，则绝对误差为 $|K_{\text{new}} - K_{\text{old}}|$。
- **相对误差：**
  指绝对误差与某一参考值（如当前迭代值或上一次迭代值）的比值。例如，以当前迭代值为分母的相对误差可表示为 $\left|\frac{K_{\text{new}} - K_{\text{old}}}{K_{\text{new}}}\right|$。相对误差在处理不同量级的数值时，更能反映误差的实际影响程度。

**迭代循环中的条件判断**

在 Python 中，可使用 `while` 或 `for` 循环进行迭代计算。以下为示例代码。

```python
import numpy as np
def calculate_permeability_1(Q, M, sw, rw, tolerance=1e-4):
    # 设置渗透系数初值为 1.0
    K = 1.0
    # 设置初始误差为无穷大
    error = np.inf
    # 当误差大于允许值时，继续迭代
    while error > tolerance:
        # 根据吉哈尔特经验公式计算影响半径 R
        R = 10 * sw * np.sqrt(K)
        # 根据 Dupuit 稳定流公式计算新的渗透系数 new_K
```

```
        new_K = Q / (2 * np.pi * M * sw) * np.log(R / rw)
        # 计算本次迭代的绝对误差
        error = np.abs(new_K - K)
        # 更新渗透系数，准备下一次迭代
        K = new_K
    # 返回最终计算出的渗透系数 K 和影响半径 R
    return K, R
# 示例：单组数据计算
# 给定：抽水量 320.54 m^3/d，含水层厚度 16.5 m，
#       抽水井半径 0.4 m，降深 1.16 m
Q = 320.54      # m^3/d
M = 16.5        # m
rw = 0.4        # m
sw = 1.16       # m
# 调用函数进行计算
K, R = calculate_permeability_1(Q, M, sw, rw)
# 输出结果，保留 4 位小数
print('渗透系数 K = {:.4f} m/d'.format(K))
print('影响半径 R = {:.4f} m'.format(R))
```

```
渗透系数 K = 12.3221 m/d
影响半径 R = 40.7192 m
```

在上述代码中：

- 首先将误差 `error` 初始化为一个较大值（此处使用 `np.inf` 表示无穷大），并设定了误差限 `tolerance`。
- 在 `while` 循环中，每次计算新的 $K$ 和 $R$ 后，通过计算新旧 $K$ 值的绝对误差，并与误差限进行比较。若误差小于误差限，则迭代终止。

若需采用相对误差，仅需修改 `error` 的计算方式，例如：`error = np.abs((new_K - K) / new_K)`。此外，可添加迭代计数器，当迭代次数超过预设阈值时强制退出循环，以避免无限循环。

`calculate_permeability_1` 函数仅能处理单组数据。若需以数组形式传递多组参数进行计算，需对该函数进行改写。

```
import numpy as np
# 设置显示格式：浮点数显示 4 位小数
np.set_printoptions(formatter={'float': '{:.4f}'.format})
def calculate_permeability_2(Q, M, sw, rw, tolerance=1e-4):
    # 创建与 Q 形状相同的数组，初始值全为 1.0
    K = np.ones_like(Q)
    # 初始误差设为无穷大
    error = np.inf
    # 只要数组中任何一个元素的误差大于 tolerance，就继续循环
```

```
    while np.any(error > tolerance):
        # 根据经验公式计算影响半径 R（数组运算）
        R = 10 * sw * np.sqrt(K)
        # 根据 Dupuit 公式计算新的渗透系数
        new_K = Q / (2 * np.pi * M * sw) * np.log(R / rw)
        # 计算每个位置上的绝对误差
        error = np.abs(new_K - K)
        # 更新渗透系数
        K = new_K
    return K, R
# 三组试验数据
Q = np.array([320.54, 421.63, 536.54])   # 抽水量数组
M = np.array([16.5, 16.5, 16.5])         # 含水层厚度数组
rw = np.array([0.4, 0.4, 0.4])           # 抽水井半径数组
sw = np.array([1.16, 1.60, 1.90])        # 对应的降深数组
# 调用批量计算函数
K, R = calculate_permeability_2(Q, M, sw, rw)
print('渗透系数 K =', K, 'm/d')
print('影响半径 R =', R, 'm')
```

```
渗透系数 K = [12.3221 12.5963 14.1221] m/d
影响半径 R = [40.7192 56.7859 71.4008] m
```

亦可使用 np.vectorize 将 calculate_permeability_1 向量化，得到可接受数组参数的函数 calculate_permeability_3。

```
import numpy as np
# 使用 np.vectorize 将前面定义的单组函数转换为可处理数组的函数
calculate_permeability_3 = np.vectorize(calculate_permeability_1)
# 准备数据
Q = np.array([320.54, 421.63, 536.54])
M = np.array([16.5, 16.5, 16.5])
rw = np.array([0.4, 0.4, 0.4])
sw = np.array([1.16, 1.60, 1.90])
# 调用向量化后的函数，返回两个数组 K 和 R
K, R = calculate_permeability_3(Q, M, sw, rw, tolerance=1e-4)
np.set_printoptions(formatter={'float': '{:.4f}'.format})
print('渗透系数 K =', K, 'm/d')
print('影响半径 R =', R, 'm')
```

```
渗透系数 K = [12.3221 12.5963 14.1221] m/d
影响半径 R = [40.7192 56.7859 71.4008] m
```

**练习 1**

某潜水含水层完整井进行稳定流抽水试验，已知潜水面初始水位高度为 43.6 m，抽水孔半径为 0.15 m，降深为 2.8 m，稳定抽水量为 2380 $m^3/d$，试求潜水含水层渗透系数 $K$ 及影响半径 $R$（计算精度要求达到 $10^{-4}$）。

# 实验 2 交互式配线法求参

井流模型的解通常具有如下形式：

$$\frac{4\pi Ts}{Q} = W(u,\cdots) \tag{2.1}$$

其中 $W(u,\cdots)$ 为无量纲标准井函数。根据 $s$ 与 $W(u,\cdots)$、$t$ 与 $1/u$ 之间的对应关系，在双对数坐标系下，观测数据 $(s,t)$ 的曲线形态与井函数标准曲线相同，通过匹配后的坐标位移量即可求得相应的水文地质参数。配线法的基本步骤为：首先绘制标准曲线，然后移动观测数据曲线，根据水平与垂直方向的位移量确定参数 $T$ 和 $S$。

与 Theis 标准曲线不同，Hantush-Jacob 标准曲线为一族曲线，每条曲线对应一个特定的越流因素 $B$。配线时需先选定一条标准曲线以确定 $B$，再移动观测数据曲线使其尽可能贴近该标准曲线，读取位移量后计算 $T$ 和 $S$。标准曲线可通过教材中的标准函数表绘制，亦可使用本指导书附录提供的 Theis、Hantush-Jacob 井函数程序计算。

## Theis 公式的配线法

对 Theis 公式及其 $u$ 表达式取对数，可得：

$$\begin{aligned} \lg s + \lg\frac{4\pi T}{Q} &= \lg W(u) \\ \lg\frac{t}{r^2} + \lg\frac{4T}{S} &= \lg\frac{1}{u} \end{aligned} \tag{2.2}$$

由此可见，在双对数坐标系中，$s$ - $t/r^2$ 曲线的形状与 $W(u)$ - $1/u$ 标准曲线一致。根据垂直位移可计算 $T$，再结合水平位移可计算 $S$。记垂直位移为 $\Delta(\lg s) = \lg\frac{4\pi T}{Q}$，水平位移为 $\Delta(\lg\frac{t}{r^2}) = \lg\frac{4T}{S}$，则参数计算公式为：

$$T = \frac{Q}{4\pi}10^{\Delta(\lg s)}, \quad S = 4T\ 10^{-\Delta(\lg\frac{t}{r^2})} \tag{2.3}$$

该方法相当于固定标准曲线，移动 $s$ - $t/r^2$ 观测曲线与之匹配。

**例 2**

某承压含水层抽水试验，含水层的顶、底板绝对隔水，抽水时侧向边界尚无影响。抽水井为完整井，抽水量为 60 $m^3$/ h。1 号观测孔距抽水孔 43 m，2 号观测孔距抽水孔 125 m，观测资料如下表。试用 1 号观测孔数据求导水系数和贮水系数。

Table 3: 抽水试验数据

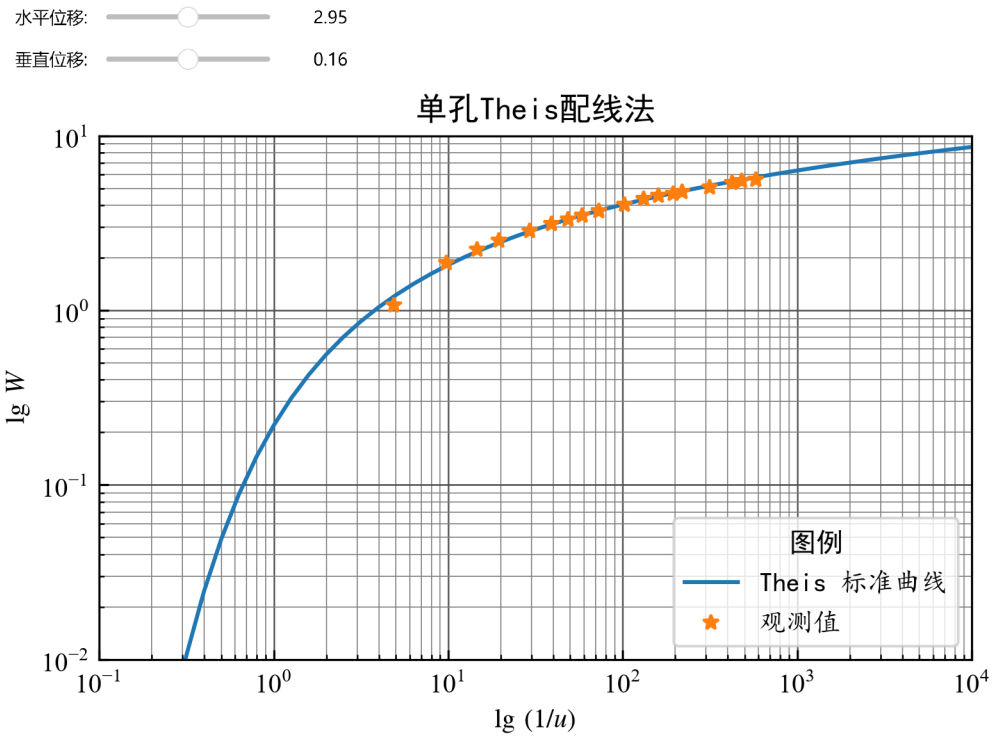
| t(min) | 1 号孔 s(m) | 2 号孔 s(m) | t(min) | 1 号孔 s(m) | 2 号孔 s(m) |
|---|---|---|---|---|---|
| 10 | 0.73 | 0.16 | 210 | 2.77 | 1.55 |
| 20 | 1.28 | 0.48 | 270 | 2.99 | 1.70 |
| 30 | 1.53 | 0.54 | 330 | 3.10 | 1.83 |
| 40 | 1.72 | 0.65 | 400 | 3.20 | 1.89 |
| 60 | 1.96 | 0.75 | 450 | 3.26 | 1.98 |
| 80 | 2.14 | 1.00 | 645 | 3.47 | 2.17 |
| 100 | 2.28 | 1.12 | 870 | 3.68 | 2.38 |
| 120 | 2.39 | 1.22 | 990 | 3.77 | 2.46 |
| 150 | 2.54 | 1.36 | 1185 | 3.85 | 2.54 |

**Python 程序**

```python
import numpy as np
from scipy.special import exp1      # 计算 Theis 井函数
from ipywidgets import interact, fixed
import ipywidgets as widgets
import matplotlib.pyplot as plt
plt.rcParams['font.family'] = ['Times New Roman']
plt.rcParams['mathtext.fontset'] = 'stix'  # 显示数学公式（STIX 字体）
plt.rcParams['axes.unicode_minus'] = False  # 解决中文负号不能正确显示的问题
plt.rcParams['xtick.direction'] = 'in'
plt.rcParams['ytick.direction'] = 'in'
# 设置 NumPy 数组的显示格式
np.set_printoptions(precision=4)
# ------------------------------------------------------------
# 函数: 绘制 Theis 配线图并计算参数
# dx, dy : 水平、垂直方向的对数位移量
# tr2    : 观测数据中的 t/r^2 值数组
# s      : 降深数组
# Q      : 抽水量 (m^3/min)
# ------------------------------------------------------------
def Theis_fit(dx, dy, tr2, s, Q, title_suffix=""):
    # 创建图形和坐标轴
    fig, ax = plt.subplots( )
    # 设置坐标轴: 双对数坐标, 范围固定
    ax.set(xscale="log", yscale="log",
```

```
            xlim=(1.0e-1, 1.0e4), ylim=(1.0e-2, 10.0),
            xlabel=r"$\lg\,(1/u)$", ylabel=r"$\lg\,W$")
    # 显示网格（主线和次线）
    ax.grid(True, which="major", linestyle="-", linewidth=0.6, color="#505050")
    ax.grid(True, which="minor", linestyle="-", linewidth=0.4, color="#808080")
    # 设置坐标轴刻度比例相同，使网格边长相等
    ax.set_aspect('equal')
    # 生成用于绘制标准曲线的横坐标（1/u 的值）
    x_curve = 10**np.linspace(-1, 4, 51)   # 从 0.1 到 10000 的对数等间距点
    # 计算标准曲线纵坐标：W(u) = exp1(1/u)
    y_curve = exp1(1 / x_curve)
    ax.plot(x_curve, y_curve, label="Theis 标准曲线", linestyle="-")
    # 绘制移动后的观测数据：横坐标乘以 10^dx，纵坐标乘以 10^dy
    ax.plot(tr2 * 10**dx, s * 10**dy, "*", label="观测值")
    # 添加图例（位置右下角）
    plt.legend(
        loc='lower right', prop={"family": "KaiTi"}, title="图例",
        title_fontproperties={'family': 'SimHei'}
        )
    ax.set_title(f"{title_suffix}", fontfamily='SimHei')
    plt.tight_layout()   # 自动调整布局
    plt.savefig(f"fig_files/{title_suffix}.png",dpi=300,bbox_inches='tight')
    plt.pause(0.01)      # 暂停更新并显示图形
    # 根据位移量计算参数
    # 由配线原理：T = (Q / (4π)) * 10^(Δlg s)
    T = 10**dy * Q / (4 * np.pi)
    # S = 4T * 10^(-Δlg(t/r^2))
    S = 10**(-dx) * 4 * T
    print(f'导水系数：T = {T:.4f} m^2/min')
    print(f'储水系数：S = {S:.4e}')
# ------------------------------------------------------------
# 准备数据（例 2：单孔观测数据）
# ------------------------------------------------------------
Q = 60.0 / 60        # 抽水量，单位转换为 m^3/min
r1 = 43.0            # 观测孔距离，单位 m
t1 = np.array([10, 20, 30, 40, 60, 80, 100, 120, 150, 210,
               270, 330, 400, 450, 645, 870, 990, 1185])   # 时间，分钟
s1 = np.array([0.73, 1.28, 1.53, 1.72, 1.96, 2.14, 2.28,
               2.39, 2.54, 2.77, 2.99, 3.10, 3.20, 3.26,
```

```
                3.47, 3.68, 3.77, 3.85])   # 降深, 米
# 计算 t/r^2 值
tr2 = t1 / r1**2
# 使用公共函数估算初始参数, 获得初始位移量
T_est, S_est, _, _ = initial_parameters(t1, s1, Q, r1)
# 位移公式:
#   dx = lg(4T / S)
#   dy = lg(4 T / Q)
dx_init = np.log10(4 * T_est / S_est)
dy_init = np.log10(4 * np.pi * T_est / Q)
# 创建水平位移滑块
dx_slider = widgets.FloatSlider(
    value=dx_init, min=dx_init - 0.5,
    max=dx_init + 0.5, step=0.01,
    description="水平位移:",
    continuous_update=False, readout_format='.2f'
    )
# 创建垂直位移滑块
dy_slider = widgets.FloatSlider(
    value=dy_init, min=dy_init - 0.5,
    max=dy_init + 0.5, step=0.01,
    description="垂直位移:",
    continuous_update=False, readout_format='.2f'
    )
# 启动交互: 拖动滑块时自动更新图形和参数
interact(
    Theis_fit,
    dx=dx_slider, dy=dy_slider,
    tr2=widgets.fixed(tr2),   # 固定参数
    s=widgets.fixed(s1),
    Q=widgets.fixed(Q),
    title_suffix="单孔 Theis 配线法"
    );
```




导水系数: T = 0.1163 m^2/min

储水系数: S = 5.1645e-04

**程序说明:**

上述程序既适用于单观测孔数据，亦可用于多观测孔数据，仅数据准备方式有所不同。子例程 `initial_parameters` 取数据中部的相邻两点，按 Jacob 公式计算初始参数，可使观测数据在初始状态下尽可能靠近标准曲线；`Theis_fit` 函数则负责绘制标准曲线及移动后的观测数据散点图，并计算输出参数。

程序中使用了 `ipywidgets` 库创建交互式控件。该库提供了多种独立控件，如 `IntSlider`、`FloatSlider`、`Text`、`Dropdown`、`Checkbox`、`Button`、`Output` 等。

以 `FloatSlider` 为例，其为 `ipywidgets` 中的浮点数滑块控件，通过滑块位置获取浮点数值。其参数包括 `value`（当前值）、`min`（最小值）、`max`（最大值）、`step`（步长）、`description`（控件左侧的标签文字）、`continuous_update`（拖动时是否实时触发更新，默认为 `True`）、`orientation`（滑块方向）、`readout`（是否显示当前数值，默认为 `True`）、`readout_format`（显示格式，默认为`'.2f'`，即保留 2 位小数）等。

`interact` 函数可自动为指定函数生成交互控件，其第一个参数为函数，后续参数为控件（对应函数的交互参数）；`interactive` 函数与 `interact` 类似，但返回一个交互对象，需调用 `IPython.display()` 进行显示；`fixed` 函数用于固定某个参数，使其不参与交互。

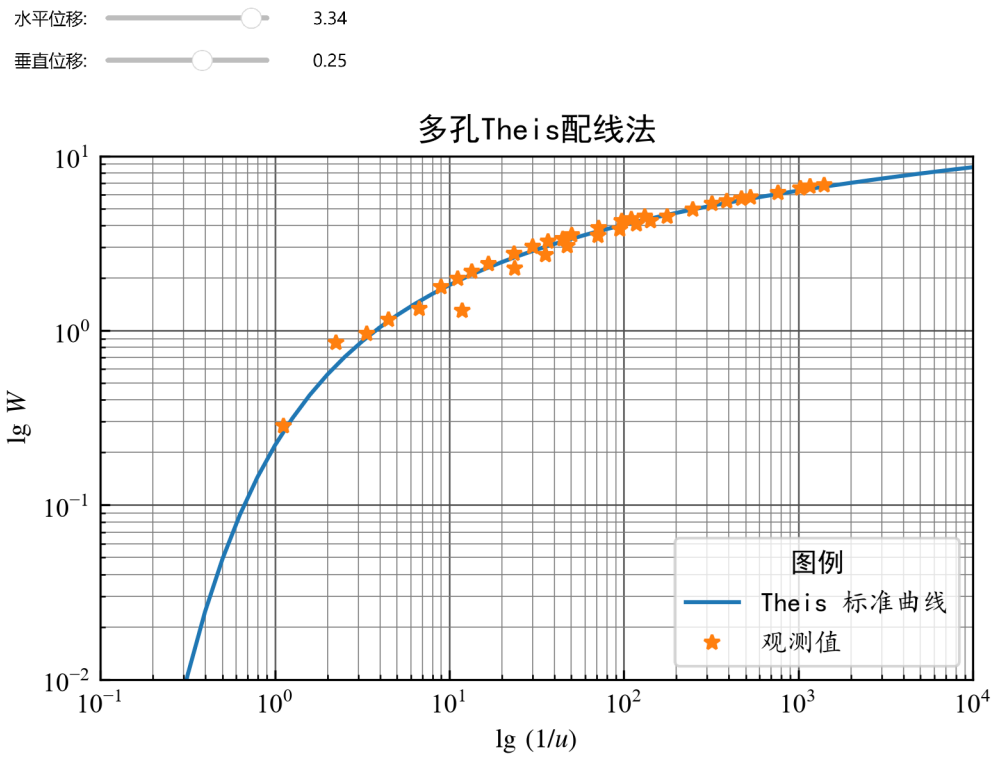
移动滑块使观测数据与标准曲线达到最佳匹配，即可得到对应的最优参数 $T$ 和 $S$。

**例 3**

根据 **例 2** 的观 1、观 2 数据求含水层水文地质参数。多个数组的拼接可使用 `np.concatenate()` 函数。

```
import numpy as np
from scipy.special import exp1
from ipywidgets import interact, widgets
import matplotlib.pyplot as plt
plt.rcParams['font.family'] = ['Times New Roman']
plt.rcParams['mathtext.fontset'] = 'stix'    # 显示数学公式（STIX 字体）
plt.rcParams['axes.unicode_minus'] = False  # 解决中文负号不能正确显示的问题
plt.rcParams['xtick.direction'] = 'in'
plt.rcParams['ytick.direction'] = 'in'
np.set_printoptions(precision=4)
# 抽水量
Q = 60.0 / 60
# 观测孔 1 数据
r1 = 43.0
t1 = np.array([10, 20, 30, 40, 60, 80, 100, 120, 150, 210,
               270, 330, 400, 450, 645, 870, 990, 1185])
s1 = np.array([0.73, 1.28, 1.53, 1.72, 1.96, 2.14, 2.28, 2.39, 2.54, 2.77,
               2.99, 3.10, 3.20, 3.26, 3.47, 3.68, 3.77, 3.85])
# 观测孔 2 数据
r2 = 140.0
t2 = np.array([10, 20, 30, 40, 60, 80, 100, 120, 150, 210,
               270, 330, 400, 450, 645, 870, 990, 1185])
s2 = np.array([0.16, 0.48, 0.54, 0.65, 0.75, 1.00, 1.12, 1.22, 1.36, 1.55,
               1.70, 1.83, 1.89, 1.98, 2.17, 2.38, 2.46, 2.54])
# 将两个孔的 t/r^2 和降深 s 拼接成一个长数组
tr2 = np.concatenate((t1 / r1**2, t2 / r2**2))
s = np.concatenate((s1, s2))
# 估计初始参数（基于第一个孔）
T_est, S_est, _, _ = initial_parameters(t1, s1, Q, r1)
dx_init = np.log10(4 * T_est / S_est)
dy_init = np.log10(4 * np.pi * T_est / Q)
# 创建滑块
dx_slider = widgets.FloatSlider(
    value=3.34, min=dx_init - 0.5,
    max=dx_init + 0.5, step=0.01,
    description=r"水平位移:",
    continuous_update=False)
dy_slider = widgets.FloatSlider(
    value=0.25, min=dy_init - 0.5,
```

```
    max=dy_init + 0.5, step=0.01,
    description=r"垂直位移:",
    continuous_update=False)
# 交互函数（与单孔相同，直接调用之前定义的 Theis_fit）
interact(
    Theis_fit, dx=dx_slider, dy=dy_slider,
    tr2=widgets.fixed(tr2), s=widgets.fixed(s), Q=widgets.fixed(Q),
    title_suffix="多孔 Theis 配线法"
    );
```




导水系数: T = 0.1415 m^2/min
储水系数: S = 2.5873e-04

**练习 2**

根据 **例 2** 的观测数据，用观 2 孔数据求参，并与例 2、例 3 的结果比较，并分析差异的原因。

**思考题**

为何使用多孔数据的配线效果可能不理想？多个求参结果应如何取舍？

## Hantush - Jacob 公式的配线法

Hantush-Jacob 公式的配线法与 Theis 公式配线法原理相同，唯一区别在于需根据特定曲线确定越流因素 $B$。

对公式：

$$s \cdot \frac{4\pi T}{Q} = W\left(u, \frac{r}{B}\right), \quad t \cdot \frac{4T}{r^2 S} = \frac{1}{u} \tag{2.4}$$

两边取对数，得：

$$\lg s + \lg \frac{4\pi T}{Q} = \lg W\left(u, \frac{r}{B}\right), \quad \lg t + \lg \frac{4T}{r^2 S} = \lg \frac{1}{u} \tag{2.5}$$

**程序的设计思路**

在标准曲线坐标系中移动 $s$ - $t$ 散点图，根据位移量计算 $(T, S)$。记坐标位移量为：

$$\Delta \lg(s) = \lg \frac{4\pi T}{Q}, \quad \Delta \lg(t) = \lg \frac{4T}{r^2 S} \tag{2.6}$$

则参数 $T$ 与 $S$ 的计算公式为：

$$T = \frac{Q}{4\pi} 10^{\Delta \lg(s)}, \quad S = \frac{4T}{r^2} 10^{-\Delta \lg(t)} \tag{2.7}$$

输入参数 $\beta = r/B$ 并绘制对应的 Hantush-Jacob 标准曲线，以最佳匹配结果确定 $\beta$。

**例 4**

对某承压含水层抽水孔进行 12 小时的定流量非稳定流抽水试验，抽水量为 528 $m^3/d$，距抽水孔 90 m 处有一观测孔，观测数据如下表所示，试确定该含水层的水文地质参数。

Table 4: 承压含水层抽水试验观测数据

| 序号 | 时间 (min) | 降深 (cm) | 序号 | 时间 (min) | 降深 (cm) |
|---|---|---|---|---|---|
| 1 | 1 | 2.5 | 9 | 50 | 24.7 |
| 2 | 2 | 3.9 | 10 | 60 | 26.4 |
| 3 | 4 | 6.1 | 11 | 90 | 30.4 |
| 4 | 6 | 8.0 | 12 | 120 | 33.0 |
| 5 | 9 | 10.6 | 13 | 150 | 35.0 |
| 6 | 20 | 16.8 | 14 | 360 | 42.6 |
| 7 | 30 | 20.0 | 15 | 550 | 44.0 |
| 8 | 40 | 22.6 | 16 | 720 | 44.5 |

**Python 程序**

```python
import numpy as np
import scipy.special as sps
from scipy.special import exp1
import ipywidgets as widgets
from ipywidgets import interact
import matplotlib.pyplot as plt
plt.rcParams['font.family'] = ['Times New Roman']
plt.rcParams['mathtext.fontset'] = 'stix'  # 显示数学公式（STIX 字体）
plt.rcParams['axes.unicode_minus'] = False  # 解决中文负号不能正确显示的问题
```

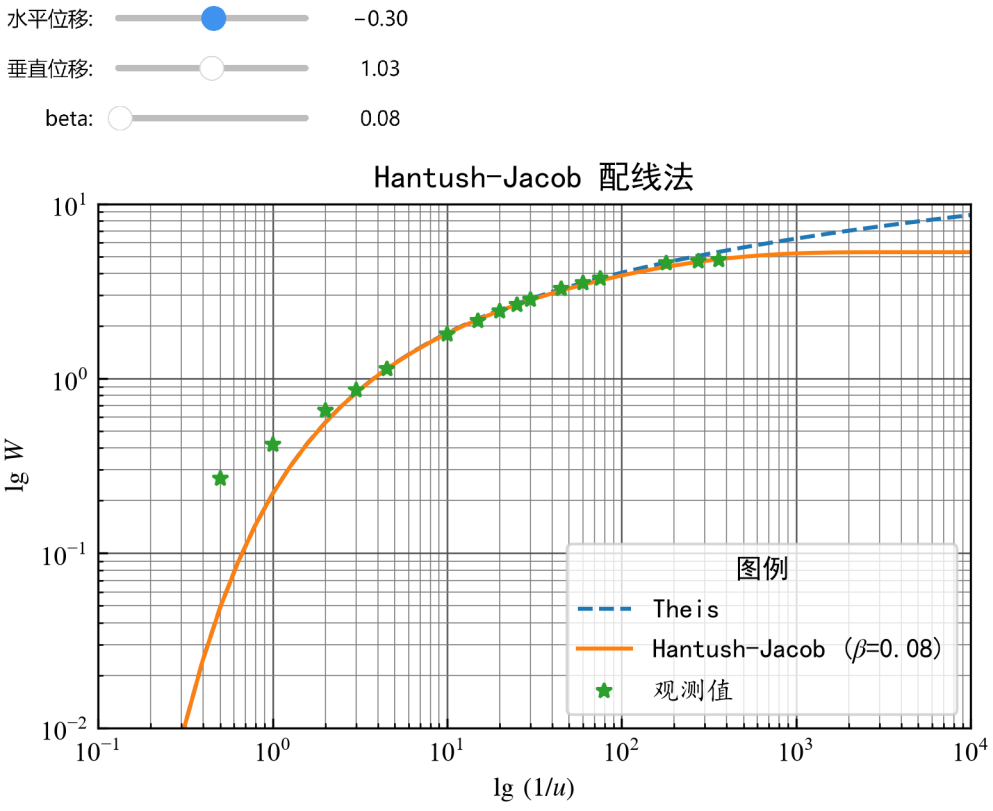
```
np.set_printoptions(precision=4)
# ------------------------------------------------------------
# 函数：计算 Hantush-Jacob 井函数 W(u, beta)
# 输入：
#   u    - 数组，u = r²S/(4Tt)
#   beta - 数组或标量，beta = r/B
# 输出：
#   W    - 井函数值（与 u 形状相同）
# 实现原理：采用级数展开，根据 beta 与 2√u 的大小关系选择不同公式
# ------------------------------------------------------------
def hantush_jacob(u, beta):
    # 定义内部函数，处理单个数值
    def wellfunc(u_val, beta_val):
        if u_val < 0:
            print('错误：u 不能为负数')
            return np.nan
        # 当 u = 0 时，对应稳定流状态，W = 2 K0(beta)
        if u_val == 0.0:
            return 2.0 * sps.k0(beta_val)

        # 根据 beta 与 2√u 的关系选择级数
        if beta_val <= 2 * np.sqrt(u_val):
            # 情况 1：beta  2√u，使用第一类级数
            W_sum = 0.0
            n = 0
            term = sps.expn(n + 1, u_val)    # n=0 时的指数积分
            while np.abs(term) > 1e-10:      # 当项小于 1e-10 时停止
                W_sum += term
                n += 1
                # 递推系数 r = (-beta²/(4u))^n / n!
                # 这里用累乘方式更新
                if n == 1:
                    r = -beta_val**2 / (4 * u_val)
                else:
                    r = r * (-beta_val**2 / (4 * u_val)) / n
                term = r * sps.expn(n + 1, u_val)
            return W_sum
        else:
            # 情况 2：beta > 2√u，使用第二类级数
```

```
            W_sum = 2.0 * sps.k0(beta_val)
            n = 0
            term = sps.expn(n + 1, beta_val**2 / (4 * u_val))
            while np.abs(term) > 1e-10:
                W_sum -= term
                n += 1
                if n == 1:
                    r = -u_val
                else:
                    r = r * (-u_val) / n
                term = r * sps.expn(n + 1, beta_val**2 / (4 * u_val))
            return W_sum
    # 将内部函数向量化，使其可以处理数组输入
    vec_wellfunc = np.vectorize(wellfunc)
    return vec_wellfunc(u, beta)
# ------------------------------------------------------------
# 绘图交互函数
# dx, dy : 时间轴和降深轴的对数位移
# beta   : 越流参数 beta = r/B
# t, s, Q, r : 观测数据
# ------------------------------------------------------------
def Hantush_Jacob_fit(dx, dy, beta, t, s, Q, r):
    # 创建图形和坐标轴
    fig, ax = plt.subplots()
    ax.set(xscale="log", yscale="log",
           xlim=(1e-1, 1e4), ylim=(1e-2, 10.0),
           xlabel=r"$\lg\,(1/u)$", ylabel=r"$\lg\,W$")
    ax.grid(True, which="major", linestyle="-", linewidth=0.6, color="#505050")
    ax.grid(True, which="minor", linestyle="-", linewidth=0.4, color="#808080")
    ax.set_aspect('equal')
    # 标准曲线的横坐标（1/u）
    x_curve = 10**np.linspace(-1, 4, 51)
    # Theis 曲线（无越流）
    ax.plot(x_curve, exp1(1/x_curve), label="Theis", linestyle="--")
    # Hantush-Jacob 曲线
    ax.plot(
        x_curve, hantush_jacob(1/x_curve, beta),
        label=rf"Hantush-Jacob ($\beta$={beta:.2f})", linestyle="-"
        )
```

```
    # 移动后的观测数据
    ax.plot(t * 10**dx, s * 10**dy, '*', label="观测值")
    plt.legend(
        loc='lower right', prop={"family": "KaiTi"}, title="图例",
        title_fontproperties={'family': 'SimHei'}
        )
    ax.set_title("Hantush-Jacob 配线法", fontfamily='SimHei')
    plt.tight_layout()   # 自动调整布局
    plt.savefig('fig_files/Hantush-Jacob 配线法.png',dpi=300,bbox_inches='tight')
    plt.pause(0.01)      # 暂停更新并显示图形
    # 根据位移量计算参数
    T = Q / (4 * np.pi) * 10**dy
    S = 4 * T / r**2 * 10**(-dx)
    print(f' 导水系数 T = {T:.4f} m^2/min')
    print(f' 储水系数 S = {S:.4e}')
    if beta > 0:
        B = r / beta
        print(f' 越流因素 B = {B:.2f} m')
    # 计算拟合残差平方和
    u = r**2 * S / (4 * T * t)
    s_calc = Q / (4 * np.pi * T) * hantush_jacob(u, beta)
    RSS = np.sum((s_calc - s)**2)
    print(f' 残差平方和 RSS = {RSS:.4e}')
# ------------------------------------------------------------
# 准备数据（例 4）
# ------------------------------------------------------------
Q = 528 / 1440      # 抽水量, m^3/min
r = 90.0            # 观测孔距离, m
t = np.array([1, 2, 4, 6, 9, 20, 30, 40, 50, 60,
              90, 120, 150, 360, 550, 720])          # 时间, min
s = np.array([2.5, 3.9, 6.1, 8.0, 10.6, 16.8, 20.0, 22.6, 24.7, 26.4,
              30.4, 33.0, 35.0, 42.6, 44.0, 44.5]) / 100   # 降深, 米
# 用公共函数估算初始参数
T_est, S_est, _, _ = initial_parameters(t, s, Q, r)
dx_init = np.log10(4 * T_est / S_est / r**2)
dy_init = np.log10(4 * np.pi * T_est / Q)
# 创建交互控件
dx_slider = widgets.FloatSlider(
    value=-0.3, min=dx_init - 0.5, max=dx_init + 0.5, step=0.01,
```

```
    description="水平位移:", continuous_update=False)
dy_slider = widgets.FloatSlider(
    value=1.03, min=dy_init - 0.5, max=dy_init + 0.5, step=0.01,
    description="垂直位移:", continuous_update=False)
beta_slider = widgets.FloatSlider(
    value=0.08, min=0.0, max=3.0, step=0.01,
    description="beta:", continuous_update=False)
# 启动交互
interact(
    Hantush_Jacob_fit,
    dx=dx_slider, dy=dy_slider, beta=beta_slider,
    t=widgets.fixed(t), s=widgets.fixed(s),
    Q=widgets.fixed(Q), r=widgets.fixed(r)
    );
```




```
导水系数: T = 0.3127 m^2/min
储水系数: S = 3.0806e-04
越流因素 B = 1125.08 m
残差平方和 RSS = 1.4630e-03
```

**程序说明**

- `hantush_jacob(u, beta)` 函数采用级数展开计算 Hantush-Jacob 井函数。

- 通过滑块 `beta_slider` 选择不同的 $\beta$ 值，实现标准曲线的切换。
- 输出残差平方和（RSS）用于辅助判断匹配质量。

**练习 3**

某河阶地上部为潜水层，其下为 2 m 厚弱透水的亚砂土，再下为 1.5 m 厚的中、粗砂层（承压含水层）。水源地抽取承压水，以 T32 号孔进行非稳定抽水试验，距其 197 m 处有 T31 孔观测，抽水量为 $Q = 69.1\ m^3/h$，观测孔水位降深值如下表所示，试用配线法求取水文地质参数。

Table 5: 某越流含水层抽水试验 T31 观测孔降深资料

| t (min) | 降深 s(m) | t (min) | 孔降深 s(m) | t (min) | 孔降深 s(m) |
|---|---|---|---|---|---|
| 1 | 0.05 | 60 | 0.575 | 300 | 0.763 |
| 4 | 0.054 | 75 | 0.62 | 330 | 0.77 |
| 7 | 0.12 | 90 | 0.64 | 360 | 0.77 |
| 10 | 0.175 | 120 | 0.685 | 390 | 0.785 |
| 15 | 0.26 | 150 | 0.725 | 420 | 0.79 |
| 20 | 0.33 | 180 | 0.735 | 450 | 0.792 |
| 25 | 0.383 | 210 | 0.755 | 480 | 0.794 |
| 30 | 0.425 | 240 | 0.76 | 510 | 0.795 |
| 45 | 0.52 | 270 | 0.76 | 540 | 0.796 |

# 实验 3 交互式拐点法求参

Hantush-Jacob 公式表达式为：

$$s = \frac{Q}{4\pi T}\int_u^\infty \frac{1}{y}e^{-y-\frac{\beta^2}{4y}}\,dy = \frac{Q}{4\pi T}W(u,\beta) \tag{3.1}$$

式中，$u = \frac{r^2 S}{4Tt}$，$\beta = \frac{r}{B}$。

在单对数坐标系下，$s$ - $\lg t$ 曲线存在拐点，且拐点处具有如下性质：

**拐点 $P$ 的性质**

- **坐标与降深**

$$t_p = \frac{SBr}{2T},\quad s_p = \frac{1}{2}s_{max} = \frac{Q}{4\pi T}K_0\left(\frac{r}{B}\right) \tag{3.2}$$

- **切线斜率**

$$i_p = \frac{2.3Q}{4\pi T}e^{-\frac{r}{B}} \tag{3.3}$$

- **$s_p$、$i_p$ 的关系**

$$2.3\frac{s_p}{i_p} = e^{\frac{r}{B}}K_0\left(\frac{r}{B}\right) \tag{3.4}$$

利用上述拐点性质可求取水文地质参数。

**设计思路**

1. 调整参数确定拐点坐标与降深 $(t_p, s_p)$，最大降深 $s_{\max}$ 可采用外推法确定。
2. 调整参数确定拐点切线斜率 $i_p$。
3. 使用二分法求解方程 $e^{\frac{r}{B}}K_0\left(\frac{r}{B}\right) - 2.3\frac{s_p}{i_p} = 0$ 得到 $\frac{r}{B}$ 的值。由于函数 $f(x) = e^x K_0(x)$ 在区间 $[0.01, 5]$ 上单调递减（$f'(x) = e^x(K_0(x) - K_1(x)) < 0$），二分法求解有效。
4. 计算参数：

$$B = \frac{r}{\left[\frac{r}{B}\right]},\quad T = \frac{2.3Q}{4\pi i_p}e^{-\frac{r}{B}},\quad S = \frac{2Tt_p}{Br}$$

5. 验证：为避免确定 $s_{\max}$ 的主观性，可采用 Hantush-Jacob 公式进行理论验证。

与交互式配线法类似，本实验采用 `widgets.FloatSlider` 控件交互式调整拐点位置与切线斜率，拐点降深可通过输入框设定。程序中所需的 Hantush-Jacob 井函数计算可复用已有程序。

**例 5**

根据 **例 4** 的数据，采用拐点法求取水文地质参数。

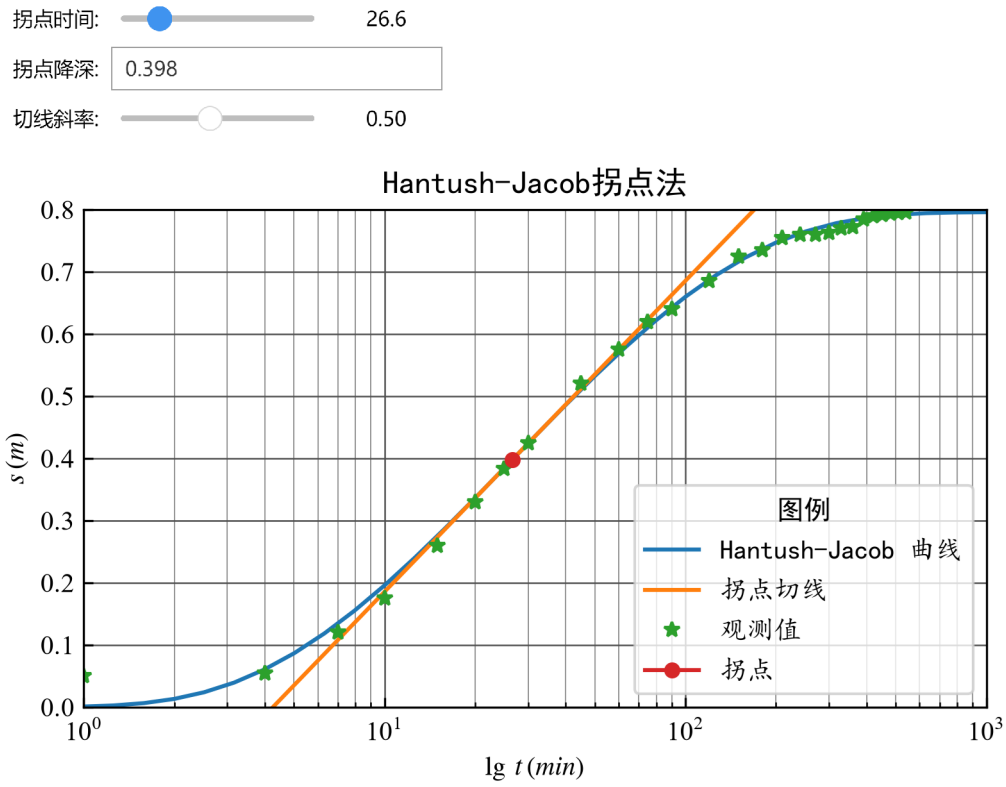
**Python 程序**

```python
import math
import numpy as np
import scipy.special as sps
from scipy.optimize import bisect
import ipywidgets as widgets
from ipywidgets import interact
import matplotlib.pyplot as plt
plt.rcParams['font.family'] = ['Times New Roman']
plt.rcParams['mathtext.fontset'] = 'stix'  # 显示数学公式（STIX 字体）
plt.rcParams['axes.unicode_minus'] = False  # 解决中文负号不能正确显示的问题
np.set_printoptions(precision=4)
# 复用前面的 hantush_jacob 函数
# 为了完整性，这里再次给出简单版本（实际运行时需确保函数存在）
def hantush_jacob(u, beta):
    def wellfunc(u_val, beta_val):
        if u_val < 0:
            return np.nan
        if u_val == 0:
            return 2.0 * sps.k0(beta_val)
        if beta_val <= 2 * np.sqrt(u_val):
            W = 0.0
            n = 0
            term = sps.expn(1, u_val)
            while np.abs(term) > 1e-10:
                W += term
                n += 1
                if n == 1:
                    r = -beta_val**2 / (4 * u_val)
                else:
                    r *= -beta_val**2 / (4 * u_val) / n
                term = r * sps.expn(n+1, u_val)
            return W
        else:
            W = 2.0 * sps.k0(beta_val)
            n = 0
            term = sps.expn(1, beta_val**2/(4*u_val))
            while np.abs(term) > 1e-10:
                W -= term
                n += 1
```

```
                if n == 1:
                    r = -u_val
                else:
                    r *= -u_val / n
                term = r * sps.expn(n+1, beta_val**2/(4*u_val))
            return W
    return np.vectorize(wellfunc)(u, beta)
# ------------------------------------------------------------
# 拐点法绘图与计算函数
# tp, sp  : 拐点的时间和降深
# slope   : 拐点处的切线斜率 (s / log10(t))
# t, s, Q, r : 观测数据
# ------------------------------------------------------------
def inflection_point_fit(tp, sp, slope, t, s, Q, r):
    fig, ax = plt.subplots()
    # 计算坐标范围
    imin = math.floor(math.log10(min(t)))
    imax = math.ceil(math.log10(max(t)))
    xmin = 10**imin
    xmax = 10**imax
    ymin = 0.0
    ymax = math.ceil(max(s * 10)) / 10
    ax.set(xscale="log", xlim=(xmin, xmax), ylim=(ymin, ymax),
           xlabel=r'$\lg\,t\,(min)$', ylabel=r'$s\,(m)$')
    ax.grid(True, which="major", linestyle="-", linewidth=0.6, color="#505050")
    ax.grid(True, which="minor", linestyle="-", linewidth=0.4, color="#808080")
    # 用二分法求解方程: exp(beta)*K0(beta) = 2.3 * sp / slope
    # 定义需要求解的函数 f(beta)
    def f(beta):
        return np.exp(beta) * sps.k0(beta) - 2.3 * sp / slope
    beta = bisect(f, 0.01, 5)   # 在区间 [0.01, 5] 上求根
    # 生成用于绘制理论曲线的时间点
    t_plot = 10**np.linspace(imin, imax, (imax - imin)*10 + 1)
    # 计算参数
    T = 2.3 * Q * np.exp(-beta) / (4 * np.pi * slope)
    S = 2 * T * tp * beta / r**2
    B = r / beta
    # 计算 Hantush-Jacob 理论降深（用于对比）
    u = 0.25 * r**2 * S / (T * t_plot)
```

```
    s_theory = 0.25 * Q * hantush_jacob(u, beta) / (np.pi * T)
    ax.plot(t_plot, s_theory, label="Hantush-Jacob 曲线")
    # 绘制拐点切线（点斜式）
    ax.plot(t_plot, sp + slope * np.log10(t_plot / tp), label="拐点切线")
    # 绘制观测数据
    ax.plot(t, s, '*', label="观测值")
    # 标记拐点位置
    ax.plot(
        tp, sp, marker="o", markersize=5,
        label=f"拐点"
        )
    plt.legend(
        loc='lower right', prop={"family": "KaiTi"}, title="图例",
        title_fontproperties={'family': 'SimHei'}
        )
    ax.set_title("Hantush-Jacob 拐点法", fontfamily='SimHei')
    plt.tight_layout()   # 自动调整布局
    plt.savefig('fig_files/拐点法.png',dpi=300,bbox_inches='tight')
    plt.pause(0.01)      # 暂停更新并显示图形
    # 输出计算结果
    print(f'导水系数 T = {T:.4f} m^2/min')
    print(f'储水系数 S = {S:.4e}')
    print(f'越流因素 B = {B:.4f} m')
    plt.close(fig)   # 关闭当前图形
# ------------------------------------------------------------
# 准备数据（例 5，即前面练习 3 的数据）
# ------------------------------------------------------------
Q = 69.1 / 60       # 抽水量 m^3/min（69.1 m^3/h ÷ 60）
r = 197.0           # 观测孔距离 m
t = np.array(
    [1, 4, 7, 10, 15, 20, 25, 30, 45, 60, 75, 90, 120, 150, 180, 210,
    240, 270, 300, 330, 360, 390, 420, 450, 480, 510, 540])   # min
s = np.array(
    [0.05, 0.054, 0.12, 0.175, 0.26, 0.33, 0.383, 0.425, 0.52, 0.575,
    0.62, 0.64, 0.685, 0.725, 0.735, 0.755, 0.76, 0.76, 0.763, 0.77,
    0.772, 0.785, 0.79, 0.792, 0.794, 0.795, 0.796])   # m
# 初始拐点估计：取最大降深的一半处
sp_initial = 0.5 * s[-1]
# 找到第一个大于 sp_initial 的索引
```

```
idx = np.where(s > sp_initial)[0][0]
tp_initial = t[idx]
# 计算该点附近的斜率（与下一个点）
slope_initial = (s[idx] - s[idx+1]) / np.log10(t[idx] / t[idx+1])
# 创建交互控件
tp_min = tp_initial * 10**(-0.5)
tp_max = tp_initial * 10**(0.5)
tp_step = (tp_max - tp_min) / 50
tp_slider = widgets.FloatSlider(
    value=26.6, min=tp_min, max=tp_max,
    step=tp_step, description='拐点时间:',
    continuous_update=False, readout_format='.1f'
    )
sp_text = widgets.FloatText(
    value=0.398, description='拐点降深:',
    step=0.001, continuous_update=False
    )
slope_slider = widgets.FloatSlider(
    value=0.5, min=slope_initial-0.5,
    max=slope_initial+0.5, step=0.01,
    description='切线斜率:', continuous_update=False,
    readout_format='.2f'
    )
interact(
    inflection_point_fit,
    tp=tp_slider, sp=sp_text, slope=slope_slider,
    t=widgets.fixed(t), s=widgets.fixed(s),
    Q=widgets.fixed(Q), r=widgets.fixed(r)
    );
```




导水系数: T = 0.3093 m^2/min

储水系数: S = 1.3134e-04

越流因素 B = 635.90 m

**程序说明**

- `bisect` 函数用于求解方程以得到 $\beta$ 值。
- 拐点切线斜率和位置可通过滑块交互调整，程序将自动更新参数计算。

**练习 4**

根据 **例 4** 的数据，采用拐点法求取水文地质参数。

# 实验 4 交互式直线图解法求参

以 $s$ - $\lg t$ 关系为例，Jacob 近似公式可表示为：

$$s \sim \frac{0.183Q}{T} \lg t + \frac{0.183Q}{T} \lg \frac{2.25T}{r^2 S} \tag{4.1}$$

将其改写为点斜式：

$$s \sim i \lg x - i \lg x_0 \tag{4.2}$$

式中，$i = \frac{0.183Q}{T}$ 为直线斜率，$x = t$ 为时间变量，$x_0 = \frac{r^2 S}{2.25T}$ 为时间轴上的截距（即降深为零时对应的时间 $t_0$）。

通过调整截距 $x_0$ 与斜率 $i$ 使直线与观测数据达到最佳拟合，即可根据以下公式计算参数：

$$T = \frac{0.183Q}{i}, \quad S = \frac{2.25Tx_0}{r^2} \tag{4.3}$$

**例 6**

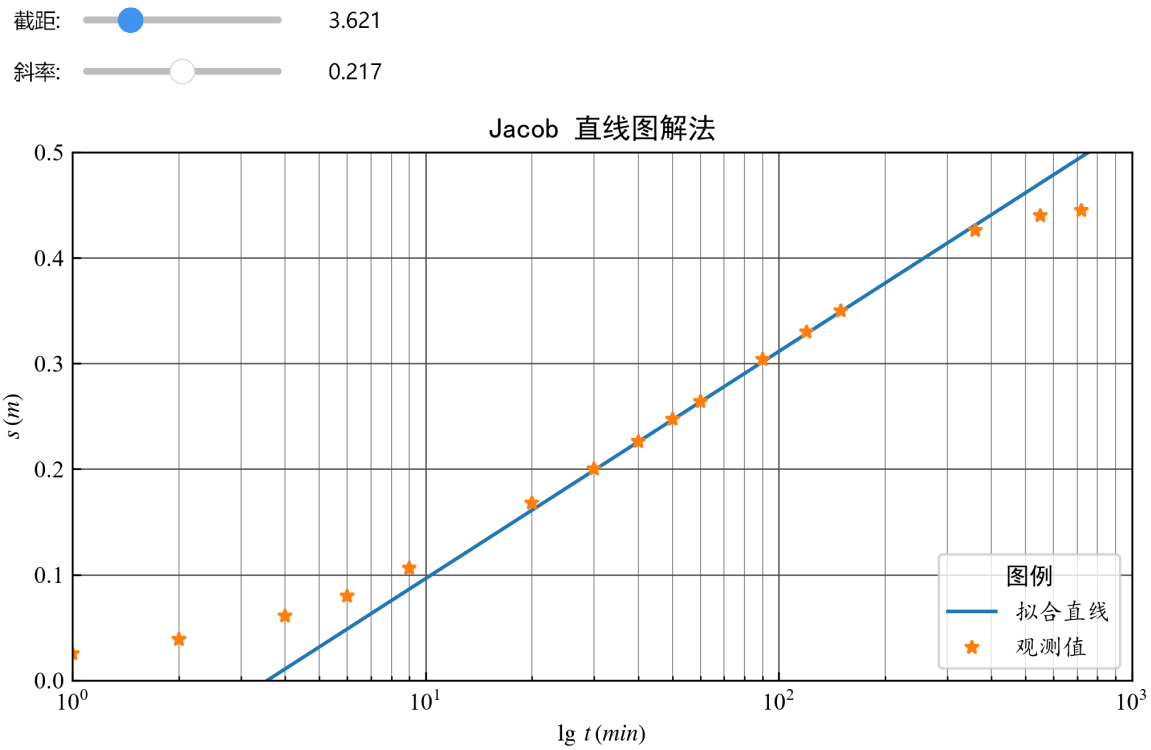
根据 **例 4** 的数据用直线图解法求水文地质参数。

**Python 程序**

```python
import numpy as np
import math
import ipywidgets as widgets
from ipywidgets import interact
import matplotlib.pyplot as plt
plt.rcParams['font.family'] = ['Times New Roman']
plt.rcParams['mathtext.fontset'] = 'stix'  # 显示数学公式（STIX 字体）
plt.rcParams['axes.unicode_minus'] = False  # 解决中文负号不能正确显示的问题
np.set_printoptions(precision=4)
# ------------------------------------------------------------
# 直线图解法绘图函数
# x0 : 直线在时间轴上的截距（即 s=0 对应的 t0）
# slope : 直线的斜率 (s / log10(t))
# t, s : 观测数据
# Q, r : 抽水量和观测距离
# ------------------------------------------------------------
def line_fit(x0, slope, t, s, Q, r):
```

```
    fig, ax = plt.subplots()
    # 确定坐标范围
    ymin = 0.0
    ymax = math.ceil(max(s * 10 + 0.5)) / 10
    imin = math.floor(math.log10(min(t)))
    imax = math.ceil(math.log10(max(t)))
    xmin = 10**imin
    xmax = 10**imax
    ax.set(xscale="log", xlim=(xmin, xmax), ylim=(ymin, ymax),
           xlabel=r'$\lg\,t\,(min)$', ylabel=r'$s\,(m)$')
    ax.grid(True, which="major", linestyle="-", linewidth=0.6, color="#505050")
    ax.grid(True, which="minor", linestyle="-", linewidth=0.4, color="#808080")
    # 生成绘制直线用的时间点
    t_line = np.linspace(xmin, xmax, 50)
    # 直线方程: s = slope * (log10(t) - log10(x0))
    s_line = slope * np.log10(t_line / x0)
    ax.plot(t_line, s_line, label="拟合直线")
    ax.plot(t, s, '*', label="观测值")
    plt.legend(
        loc='lower right', prop={"family": "KaiTi"}, title="图例",
        title_fontproperties={'family': 'SimHei'}
        )
    ax.set_title("Jacob 直线图解法", fontfamily='SimHei')
    plt.tight_layout()   # 自动调整布局
    plt.savefig('fig_files/Jacob 交互直线图解法.png',dpi=300,bbox_inches='tight')
    plt.pause(0.01)      # 暂停更新并显示图形
    # 计算参数
    T = 0.183 * Q / slope
    S = 2.25 * T * x0 / r**2
    print(f' 导水系数 T = {T:.4f} m^2/min')
    print(f' 储水系数 S = {S:.4e}')
    plt.close(fig)
# ------------------------------------------------------------
# 准备数据（例 4 数据）
# ------------------------------------------------------------
Q = 528 / 1440      # m^3/min
r = 90.0            # m
t = np.array([1, 2, 4, 6, 9, 20, 30, 40, 50, 60,
              90, 120, 150, 360, 550, 720])      # min
```

```
s = np.array([2.5, 3.9, 6.1, 8.0, 10.6, 16.8, 20.0,
              22.6, 24.7, 26.4, 30.4, 33.0, 35.0,
              42.6, 44.0, 44.5]) / 100   # m
# 用公共函数估算初始参数
T_est, S_est, x0_est, slope_est = initial_parameters(t, s, Q, r)
# 创建滑块控件
x0_min = x0_est * 10**(-0.5)
x0_max = x0_est * 10**(0.5)
x0_step = (x0_max - x0_min) / 50
x0_slider = widgets.FloatSlider(
    value=3.564, min=x0_min, max=x0_max, step=x0_step,
    description=' 截距:', continuous_update=False,
    readout_format='.3f'
    )
slope_slider = widgets.FloatSlider(
    value=0.215, min=slope_est-0.5, max=slope_est+0.5, step=0.01,
    description=' 斜率:', continuous_update=False, readout_format='.3f'
    )
interact(
    line_fit,
    x0=x0_slider, slope=slope_slider,
    t=widgets.fixed(t), s=widgets.fixed(s),
    Q=widgets.fixed(Q), r=widgets.fixed(r)
    );
```




导水系数: `T = 0.3121 m^2/min`

储水系数: `S = 3.0897e-04`

**程序说明**

- `line_fit` 函数通过滑块交互式调整截距 $x_0$ 与斜率 $i$，实现直线与观测数据点的最佳拟合。
- 初始参数由 `initial_parameters` 函数估算，确保拟合直线大致经过数据点。
- 要注意数据是否满足 Jacob 近似公式的要求，即 $u < 0.1$ 或 $u < 0.01$。

**练习 5**

根据 **例 2** 的观 1、观 2 数据采用直线图解法求取水文地质参数，并与配线法结果进行对比。

**思考题**

1. 对比 Theis 配线法与 Jacob 直线图解法的求参结果，分析二者差异及其原因。
2. 选择用于拟合直线的数据点时，应考虑哪些因素？
3. 尝试分别采用单观测孔数据与多观测孔数据（可使用 `np.concatenate()` 函数拼接数组）进行求参，并对结果进行比较。

# 实验 5 最小二乘法应用

## 最小二乘法原理

根据观测数据 $\{(x_i, y_i), i = 1, \cdots m\}$，求拟合直线 $y = \beta_1 + \beta_2 x$，使得误差平方和 $E(\beta_1, \beta_2) = \sum_{i=1}^{m}(y_i - \beta_1 - \beta_2 x_i)^2$ 取最小值。该问题亦可视为求解超定方程组，即确定系数 $\beta$，使 $X\beta = Y$ 在最小二乘意义下成立。式中，$X = \begin{bmatrix} 1 & \cdots & 1 \\ x_1 & \cdots & x_m \end{bmatrix}^T$，$Y = \begin{bmatrix} y_1 & \cdots & y_m \end{bmatrix}^T$，$\beta = \begin{bmatrix} \beta_1 & \beta_2 \end{bmatrix}^T$。

**一般形式**

考虑超定方程组（方程个数 $m$ 大于未知数个数 $n$）：

$$\sum_{j=1}^{n} x_{ij}\beta_j = y_i, \quad i = 1, \cdots m \tag{5.1}$$

其中，$m$ 为样本数，$n$ 为参数维度。其矩阵形式为：

$$X\beta = Y \tag{5.2}$$

式中，

$$X = \begin{bmatrix} x_{11} & \cdots & x_{1n} \\ \vdots & & \vdots \\ x_{m1} & \cdots & x_{mn} \end{bmatrix}, \quad \beta = \begin{bmatrix} \beta_1 \\ \vdots \\ \beta_n \end{bmatrix}, \quad Y = \begin{bmatrix} y_1 \\ \vdots \\ y_m \end{bmatrix}$$

为得到 $\beta$ 的最佳估计 $\hat{\beta}$，将问题转化为如下最优化问题：

$$\min E(\beta) = \min \|X\beta - Y\|^2$$

通过微分求极值，可得正规方程：

$$(X^T X)\beta = (X^T Y)$$

若 $X^T X$ 为非奇异矩阵，则 $\beta$ 有唯一解：

$$\hat{\beta} = (X^T X)^{-1} X^T Y \tag{5.3}$$

由此可见，求解最小二乘问题的关键在于构造相应的方程组。

`numpy` 库中的 `numpy.linalg.solve` 函数可用于求解形如 (6.2) 的方程组；`numpy.linalg.lstsq` 函数则直接实现了最小二乘法，可用于求解形如 (6.1) 的超定方程组。此外，`scipy` 库的 `scipy.linalg.lstsq` 函数亦提供最小二乘求解功能。

**例 7**

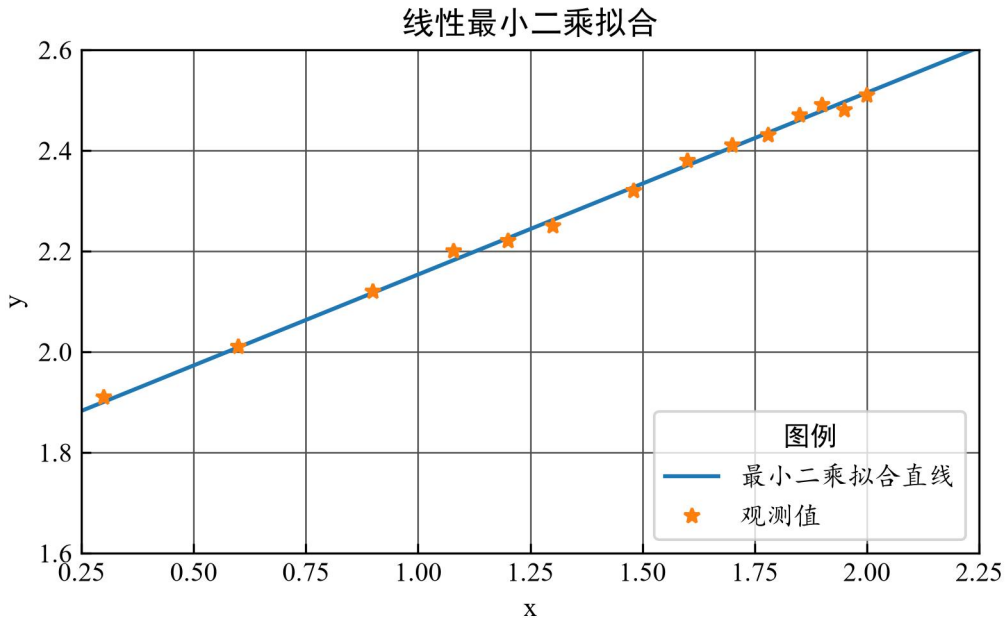
在一次模拟实验中，自变量 $x$ 的取值为 [0.00, 0.30, 0.60, 0.90, 1.08, 1.20, 1.30, 1.48, 1.60, 1.70, 1.78, 1.85, 1.90, 1.95, 2.00]，观测到的因变量 $y$ 为 [1.78, 1.91, 2.01, 2.12, 2.20, 2.22, 2.25, 2.32, 2.38, 2.41, 2.43, 2.47, 2.49, 2.48, 2.51]。根据实验分析，$y$ 与 $x$ 呈线性关系，试确定其关系表达式。

**Python 程序**

**方法 1：**构造形如 (6.2) 的正规方程组，使用 `numpy.linalg.solve` 求解。

```
import numpy as np
import matplotlib.pyplot as plt
plt.rcParams['font.family'] = ['Times New Roman']
plt.rcParams['mathtext.fontset'] = 'stix'  # 显示数学公式（STIX 字体）
plt.rcParams['axes.unicode_minus'] = False  # 解决中文负号不能正确显示的问题
np.set_printoptions(precision=4)
# 观测数据
x = np.array([0.00, 0.30, 0.60, 0.90, 1.08, 1.20, 1.30, 1.48, 1.60, 1.70,
              1.78, 1.85, 1.90, 1.95, 2.00])
y = np.array([1.78, 1.91, 2.01, 2.12, 2.20, 2.22, 2.25, 2.32, 2.38, 2.41,
              2.43, 2.47, 2.49, 2.48, 2.51])
# 构造系数矩阵 X: 第一列全 1, 第二列为 x
X = np.vstack([np.ones(len(x)), x]).T   # 形状 (15,2)
# 求解正规方程 (X^T X) beta = X^T y
beta = np.linalg.solve(X.T @ X, X.T @ y)   # @ 表示矩阵乘法
# 计算拟合值
x_val = np.linspace(0.25, 2.25, 100)
y_fit = beta[0] + beta[1] * x
y_val = beta[0] + beta[1] * x_val
# 计算残差平方和
residuals = np.sum((y - y_fit)**2)
# 绘图
fig, ax = plt.subplots()
ax.grid(True, which="major", linestyle="-", linewidth=0.6, color="#505050")
ax.set(xlim=(0.25, 2.25), ylim=(1.6, 2.6))
ax.plot(x_val, y_val, '-', label=' 最小二乘拟合直线')
ax.plot(x, y, '*', label=' 观测值')
ax.set(xlabel='x', ylabel='y')
plt.legend(
        loc='lower right', prop={"family": "KaiTi"}, title="图例",
        title_fontproperties={'family': 'SimHei'}
        )
```

```
ax.set_title(' 线性最小二乘拟合', fontfamily='SimHei')
plt.tight_layout()
plt.savefig('fig_files/最小二乘法原理.png',dpi=300,bbox_inches='tight')
plt.show()
print(f' 拟合参数: 截距 beta0 = {beta[0]:.4f}, 斜率 beta1 = {beta[1]:.4f}')
print(f' 残差平方和: {residuals:.4e}');
```




```
拟合参数: 截距 beta0 = 1.7924, 斜率 beta1 = 0.3612
残差平方和: 1.4327e-03
```

**方法 2.** 直接使用 `np.linalg.lstsq` 求解。

```
# 使用 lstsq 方法（更简洁）
np.set_printoptions(precision=4)
beta, residuals, rank, s = np.linalg.lstsq(X, y, rcond=None)
print(f' 拟合参数: 截距 beta0 = {beta[0]:.4f}, 斜率 beta1 = {beta[1]:.4f}')
print(f' 残差平方和: {residuals[0]:.4e}');
```

```
拟合参数: 截距 beta0 = 1.7924, 斜率 beta1 = 0.3612
残差平方和: 1.4327e-03
```

## 确定 Q - s 曲线类型

常见的 $Q$ - $s_w$ 曲线类型包括：

- **直线型：**

  $Q = q\ s_w$

- **抛物线型：**

$s_w = a\ Q + b\ Q^2$

- **幂函数数型：**
  $Q = q_0\ s_w^{\frac{1}{m}}$
- **对数型：**
  $Q = a + b\ \lg s_w$

为便于线性化处理，将上述类型进行变换：

- **直线型：**
  $Q = b\ s_w$ (不变，要求通过原点）$(b > 0)$
- **抛物线型：**
  $\frac{s_w}{Q} = a + b\ Q$，$(a > 0,\ b > 0)$
- **幂函数型：**
  $\lg Q = a + b\ \lg s_w$，（$b > 0$）
- **对数型：**
  $Q = a + b\ \lg s_w$ (不变)，（$a > 0,\ b > 0$ ）

根据最小二乘法原理，构造超定方程组 $X\beta = Y$，进而形成正规方程组 $A\beta = b$，其中 $A = X^T X, b = X^T Y$，使用 `numpy.linalg.solve` 求解（方法一）。

分别计算各模型预测降深与观测降深的残差平方和 $\sum(s - \hat{s})^2$，残差平方和最小者通常为最优曲线类型。

**注意：**由于以天或小时为单位的流量数值较大，建议将流量单位转换为分钟（min）进行计算。

**例 8**

在厚度为 16.50 m 的承压含水层中进行了 3 次降深的抽水试验，结果记录于下表。试确定 $Q$ - $s$ 曲线类型。

Table 6: 稳定流抽水试验数据

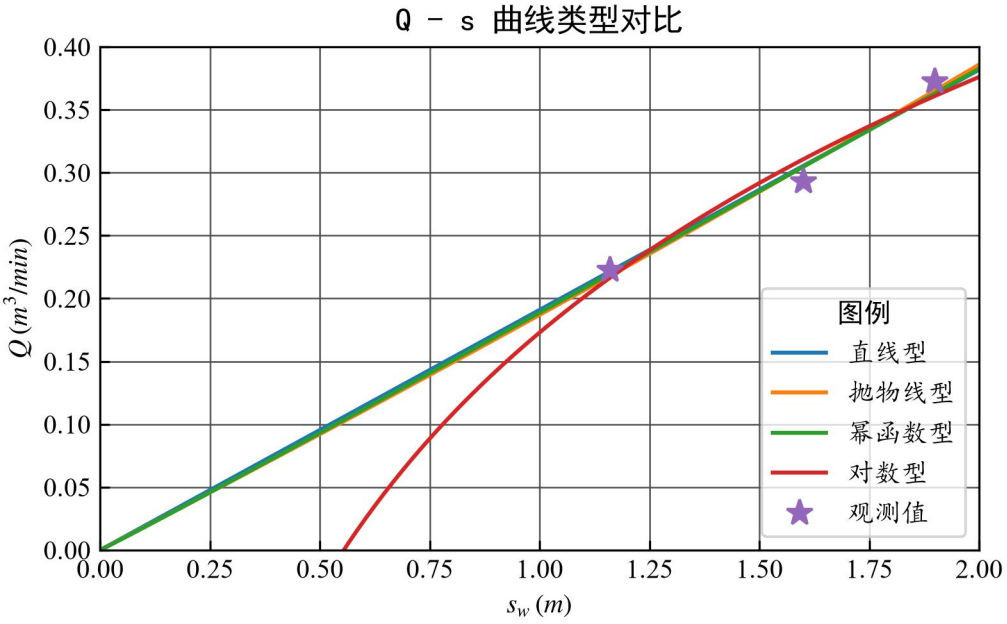
| $r_w(m)$ | $s_1(m)$ | $Q_1(m^3/d)$ | $s_2(m)$ | $Q_2(m^3/d)$ | $s_3(m)$ | $Q_3(m^3/d)$ |
|---|---|---|---|---|---|---|
| 0.40 | 1.16 | 320.54 | 1.60 | 421.63 | 1.90 | 536.54 |

**Python 程序**

```python
import numpy as np
import matplotlib.pyplot as plt
plt.rcParams['font.family'] = ['Times New Roman']
plt.rcParams['mathtext.fontset'] = 'stix'  # 显示数学公式（STIX 字体）
plt.rcParams['axes.unicode_minus'] = False  # 解决中文负号不能正确显示的问题
np.set_printoptions(precision=4)
# ------------------------------------------------------------
# 绘制 Q-s 曲线对比图
# ------------------------------------------------------------
```

```
def plot_Qs(s, Q, beta1, beta2, beta3, beta4):
    fig, ax = plt.subplots()
    # 确定坐标范围
    xmax = np.ceil(max(s * 10 + 1)) / 10
    ymax = np.ceil(max(Q * 10)) / 10
    ax.set(
        xlim=(0, xmax), ylim=(0, ymax),
        xlabel=r'$s_w\,(m)$', ylabel=r'$Q\,(m^3/min$)'
        )
    ax.grid(True, which="major", linestyle="-", linewidth=0.6, color="#505050")
    # 生成用于绘制理论曲线的降深点
    s_curve = np.linspace(1e-4, xmax, 100)
    # 直线型: Q = beta1 * s
    ax.plot(s_curve, beta1 * s_curve, label=' 直线型')
    # 抛物线型: s = beta2[0] Q + beta2[1] Q²  => 解出 Q (取正根)
    # 方程 beta2[1] Q² + beta2[0] Q - s = 0
    Q_para = (
        -beta2[0] + np.sqrt(beta2[0]**2 + 4 * beta2[1] * s_curve)
        ) / (2 * beta2[1])
    ax.plot(s_curve, Q_para, label=' 抛物线型')
    # 幂函数型: Q = 10^{beta3[0]} * s^{beta3[1]}
    Q_power = 10**beta3[0] * s_curve**beta3[1]
    ax.plot(s_curve, Q_power, label=' 幂函数型')
    # 对数型: Q = beta4[0] + beta4[1] * lg(s)
    Q_log = beta4[0] + beta4[1] * np.log10(s_curve)
    ax.plot(s_curve, Q_log, label=' 对数型')
    # 原始观测点
    ax.plot(s, Q, '*', label=' 观测值', markersize=10)
    plt.legend(
        loc='lower right', prop={"family": "KaiTi"}, title="图例",
        title_fontproperties={'family': 'SimHei'}
        )
    ax.set_title(r"Q - s 曲线类型对比", fontfamily='SimHei')
    return fig
# ------------------------------------------------------------
# 数据准备（例 8）
# ------------------------------------------------------------
Q_raw = np.array([320.54, 421.63, 536.54])   # m^3/d
s = np.array([1.16, 1.60, 1.90])             # m
```

```
Q = Q_raw / 1440    # 转换为 m^3/min
# 1. 直线型（过原点）：Q = beta1 * s，最小二乘解 beta1 = (s·Q)/(s·s)
beta1 = np.dot(s, Q) / np.dot(s, s)
# 2. 抛物线型：变换为 s/Q = a + b Q
# 构造 X = [1, Q]，Y = s/Q
X_para = np.vstack([np.ones(len(Q)), Q]).T
Y_para = s / Q
beta2 = np.linalg.solve(X_para.T @ X_para, X_para.T @ Y_para)   # beta2 = [a, b]
# 3. 幂函数型：lg Q = a + b lg s
X_power = np.vstack([np.ones(len(s)), np.log10(s)]).T
Y_power = np.log10(Q)
beta3 = np.linalg.solve(X_power.T @ X_power, X_power.T @ Y_power)
# 4. 对数型：Q = a + b lg s
X_log = np.vstack([np.ones(len(s)), np.log10(s)]).T
Y_log = Q
beta4 = np.linalg.solve(X_log.T @ X_log, X_log.T @ Y_log)
# 绘图
fig = plot_Qs(s, Q, beta1, beta2, beta3, beta4)
plt.tight_layout()
plt.savefig('fig_files/最小二乘法 Qs 经验曲线.png',dpi=300,bbox_inches='tight')
plt.show()
# 计算各模型预测降深的残差平方和（以原始降深 s 为基准）
print('各模型预测降深 s 的残差平方和：')
# 直线型：由 Q = beta1 s 得 s_pred = Q / beta1
s_pred_linear = Q / beta1
rss_linear = np.sum((s - s_pred_linear)**2)
print(f'  直线型：RSS = {rss_linear:.4e}，beta1 = {beta1:.4f}')
# 抛物线型：s_pred = a Q + b Q²
s_pred_para = beta2[0] * Q + beta2[1] * Q**2
rss_para = np.sum((s - s_pred_para)**2)
print(f'抛物线型：RSS = {rss_para:.4e}，beta2 = {beta2}')
# 幂函数型：由 lgQ = a + b lg s 得 s_pred = (Q / 10^a)^{1/b}
s_pred_power = (Q / 10**beta3[0]) ** (1 / beta3[1])
rss_power = np.sum((s - s_pred_power)**2)
print(f'幂函数型：RSS = {rss_power:.4e}，beta3 = {beta3}')
# 对数型：s_pred = 10^{(Q - a)/b}
s_pred_log = 10**((Q - beta4[0]) / beta4[1])
rss_log = np.sum((s - s_pred_log)**2)
print(f'  对数型：RSS = {rss_log:.4e}，beta4 = {beta4}')
```




各模型预测降深 s 的残差平方和：

直线型: RSS = 7.1050e-03, beta1 = 0.1909

抛物线型: RSS = 5.3899e-03, beta2 = [ 5.5048 -0.8326]

幂函数型: RSS = 6.4173e-03, beta3 = [-0.7242  1.0195]

对数型: RSS = 1.5437e-02, beta4 = [0.1729 0.6747]

**练习 6**

某煤矿对太原组含水层四、五灰进行了简易稳定流抽水试验，抽水量 $Q$、抽水井降深 $s_w$、抽水井半径 $r_w$、含水层厚度 $M$ 如下表所示。试确定抽水孔的 $Q$ - $s$ 曲线类型，并计算含水层水文地质参数（可参考实验 1）。

Table 7: 简易稳定流抽水数据表

| 孔号 | $Q(m^3/d)$ | $s_w(m)$ | $r_w(m)$ | $M(m)$ |
|---|---|---|---|---|
| 东 102 | 133.92 | 54.21 | 0.054 | 26.80 |
| | 125.28 | 44.51 | 0.054 | 26.80 |
| | 116.64 | 35.90 | 0.054 | 26.80 |
| 西 43 | 429.12 | 33.50 | 0.0445 | 34.90 |
| | 495.36 | 40.40 | 0.0445 | 34.90 |
| | 567.36 | 48.50 | 0.0445 | 34.90 |

**思考题**

1. 通过比较，抛物线型的降深残差平方和最小，问将该井的 $Q$ - $s$ 曲线确定为抛物线型是否合适？为什么？

2. 对比抛物线型的 $Q$ - $s$ 关系式与 C. E. Jacob 井损表达式可以看出，二者形式相同。若按上述示例数据计算发现井损系数 $C$ 为负值，试分析其原因。
3. 若已知含水层某钻孔的 $Q$ - $s$ 曲线类型，该类型及相关数据能否用于预测同含水层其他钻孔的降深？
4. 对于不同的 $Q$ - $s$ 曲线类型，应如何根据计算结果分析其合理性？

## Jacob 公式线性拟合

在单对数坐标系中，抽水一定时间后，实测数据点呈直线分布：

$$s = \frac{0.183Q}{T}\lg\frac{2.25T}{r^2S} + \frac{0.183Q}{T}\lg t \tag{5.4}$$

可将其改写为线性形式：

$$s = s_0 + i\lg t \tag{5.5}$$

拟合得到直线斜率 $i$ 与降深轴截距 $s_0$ 后，即可用下式计算参数：

$$T = \frac{0.183Q}{i}, \quad S = \frac{2.25T}{r^2}10^{-\frac{s_0}{i}} \tag{5.6}$$

使用最小二乘法可方便地计算出直线方程的参数。

**例 9**

根据 **例 4** 的数据，采用 Jacob 公式的最小二乘法线性拟合求取水文地质参数。

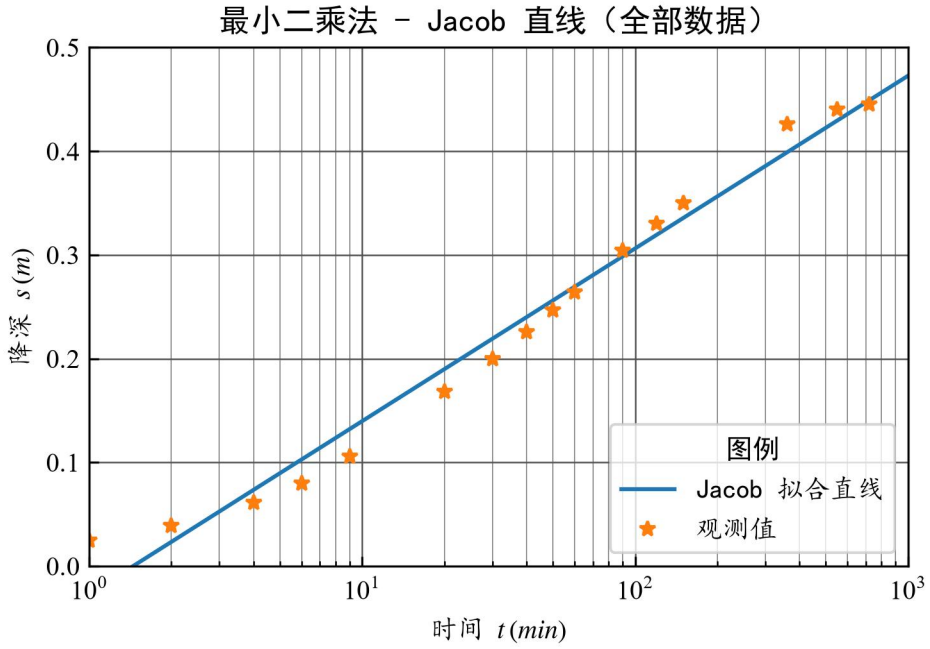
根据数据点的形态可知，前面几组观测值不满足 Jacob 公式的近似条件，后面几组数偏离直线，可能是边界条件的影响，不满足 Theis 模型的无限含水层假设。因此需要筛选数据，前 5 组与后 3 组数据不参与拟合。

**Python 程序**

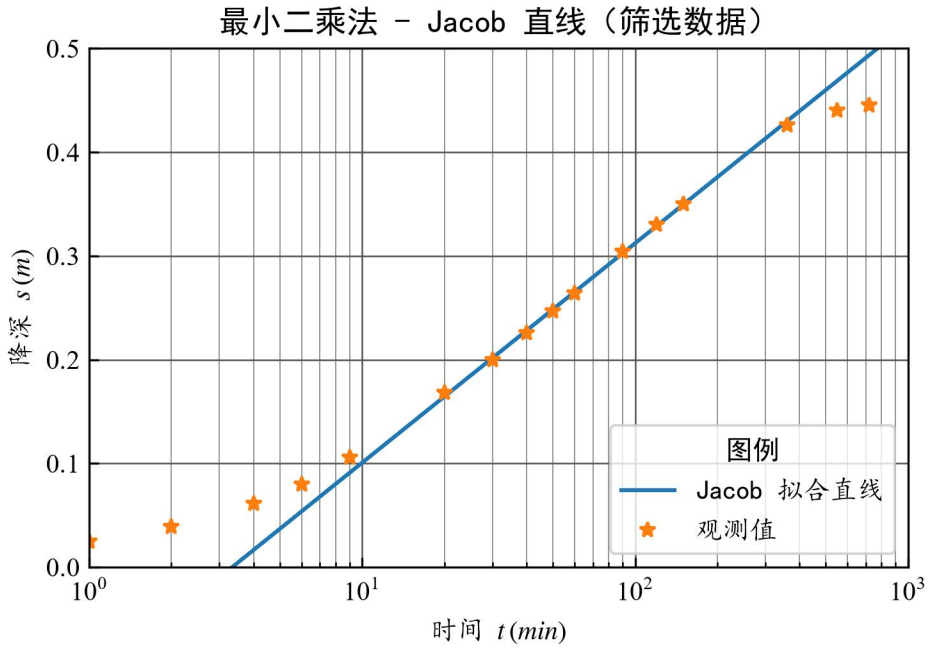
```python
import numpy as np
import matplotlib.pyplot as plt
plt.rcParams['font.family'] = ['Times New Roman']
plt.rcParams['mathtext.fontset'] = 'stix'  # 显示数学公式（STIX 字体）
plt.rcParams['axes.unicode_minus'] = False  # 解决中文负号不能正确显示的问题
np.set_printoptions(precision=4)
# ------------------------------------------------------------
# 绘制 Jacob 直线拟合结果
# ------------------------------------------------------------
def plot_jacob(t, s, beta, title_suffix=""):
    fig, ax = plt.subplots()
    ymax = np.ceil(max(s * 10)) / 10
    imin = np.floor(np.log10(min(t)))
    imax = np.ceil(np.log10(max(t)))
    xmin = 10**imin
```

```
    xmax = 10**imax
    ax.set(xscale="log", xlim=(xmin, xmax), ylim=(0, ymax))
    ax.set_xlabel(r'时间 $t\,(min)$', fontfamily="KaiTi")
    ax.set_ylabel(r'降深 $s\,(m)$', fontfamily="KaiTi")
    ax.grid(True, which="major", linestyle="-", linewidth=0.6, color="#505050")
    ax.grid(True, which="minor", linestyle="-", linewidth=0.4, color="#808080")
    # 绘制拟合直线
    t_line = np.logspace(imin, imax, 50)
    s_line = beta[0] + beta[1] * np.log10(t_line)
    ax.plot(t_line, s_line, label='Jacob 拟合直线')
    ax.plot(t, s, '*', label='观测值')
    plt.legend(
        loc='lower right', prop={"family": "KaiTi"}, title="图例",
        title_fontproperties={'family': 'SimHei'}
        )
    ax.set_title(f"最小二乘法 - Jacob 直线{title_suffix}", fontfamily='SimHei')
    plt.savefig(f"fig_files/最小二乘法 Jacob{title_suffix}.png", dpi=300)
    return fig
# 使用例 4 数据
Q = 528 / 1440      # m^3/min
r = 90.0
t = np.array([1, 2, 4, 6, 9, 20, 30, 40, 50, 60,
              90, 120, 150, 360, 550, 720])
s = np.array([2.5, 3.9, 6.1, 8.0, 10.6, 16.8, 20.0, 22.6, 24.7, 26.4,
              30.4, 33.0, 35.0, 42.6, 44.0, 44.5]) / 100
# 使用全部数据拟合
X_all = np.vstack([np.ones(len(t)), np.log10(t)]).T
beta_all = np.linalg.lstsq(X_all, s, rcond=None)[0]
fig1 = plot_jacob(t, s, beta_all, "（全部数据）")
plt.show()
# 计算参数
T_all = 0.183 * Q / beta_all[1]
S_all = 2.25 * T_all / r**2 * 10**(-beta_all[0] / beta_all[1])
print(f'全部数据: T = {T_all:.4f} m^2/min, S = {S_all:.4e}')
# 计算每个时间点的 u 值，检查是否满足 u<0.1
u_all = r**2 * S_all / (4 * T_all * t)
print('全部数据对应的 u 值: ')
print(u_all)
# 筛选数据：去掉前 5 个和后 3 个（通常 u 太小或受边界影响）
```

```
idx_selected = slice(5, -3)   # 索引从 5 到倒数第 3 个（不包括最后 3 个）
t_sel = t[idx_selected]
s_sel = s[idx_selected]
X_sel = np.vstack([np.ones(len(t_sel)), np.log10(t_sel)]).T
beta_sel = np.linalg.lstsq(X_sel, s_sel, rcond=None)[0]
fig2 = plot_jacob(t, s, beta_sel, "（筛选数据）")
plt.show()
# 计算参数
T_sel = 0.183 * Q / beta_sel[1]
S_sel = 2.25 * T_sel / r**2 * 10**(-beta_sel[0] / beta_sel[1])
print(f'筛选后数据: T = {T_sel:.4f} m^2/min, S = {S_sel:.4e}')
# 计算每个时间点的 u 值，检查是否满足 u<0.1
u_all = r**2 * S_sel / (4 * T_sel * t)
print('全部数据对应的 u 值: ')
print(u_all)
```




```
全部数据: T = 0.4030 m^2/min, S = 1.6203e-04
全部数据对应的 u 值:
[0.8142 0.4071 0.2036 0.1357 0.0905 0.0407 0.0271 0.0204 0.0163 0.0136
 0.009  0.0068 0.0054 0.0023 0.0015 0.0011]
```




筛选后数据：T = 0.3172 m^2/min, S = 2.9447e-04

全部数据对应的 u 值：

[1.8801 0.94 0.47 0.3133 0.2089 0.094 0.0627 0.047 0.0376 0.0313
0.0209 0.0157 0.0125 0.0052 0.0034 0.0026]

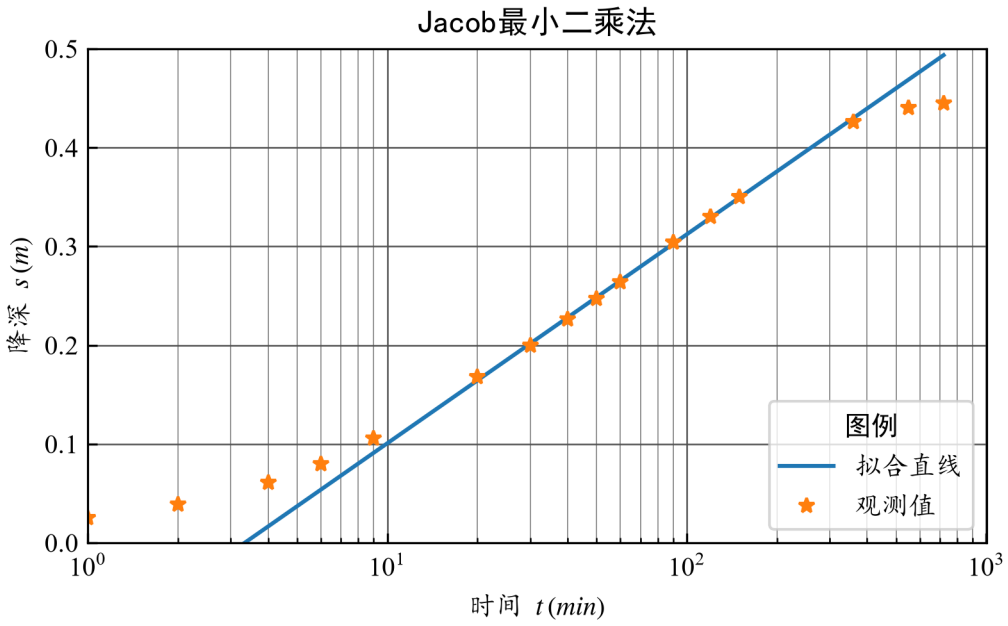
以下为交互式筛选数据的代码，更方便地筛选出符合要求的数据。

```
import numpy as np
import ipywidgets as widgets
from ipywidgets import interact
import matplotlib.pyplot as plt
plt.rcParams['font.family'] = ['Times New Roman']
plt.rcParams['mathtext.fontset'] = 'stix'  # 显示数学公式（STIX 字体）
plt.rcParams['axes.unicode_minus'] = False  # 解决中文负号不能正确显示的问题
np.set_printoptions(precision=4)
def fit(data_lims, t, s, Q, r):
    """创建并设置绘图区域"""
    fig, ax = plt.subplots()
    ymax = np.ceil(max(s * 10)) / 10
    imin = np.floor(np.log10(min(t)))
    imax = np.ceil(np.log10(max(t)))
    xmin = 10**imin
    xmax = 10**imax
    ax.set(xscale="log", xlim=(xmin, xmax), ylim=(0, ymax))
    ax.set_xlabel(r'时间 $t\,(min)$', fontfamily="KaiTi")
```

```
    ax.set_ylabel(r'降深 $s\,(m)$', fontfamily="KaiTi")
    ax.grid(True, which="major", linestyle="-", linewidth=0.6, color="#505050")
    ax.grid(True, which="minor", linestyle="-", linewidth=0.4, color="#808080")
    ax.set_title("Jacob 最小二乘法", fontfamily='SimHei')
    """更新拟合曲线并计算结果"""
    i, j = data_lims
    j += 1
    X = np.vstack([np.ones(j - i), np.log10(t[i:j])]).T
    slove = np.linalg.lstsq(X, s[i:j], rcond=None)
    beta = slove[0]
    rss = slove[1][0]
    # 绘制拟合直线
    ax.plot(t, beta[0] + beta[1] * np.log10(t), label="拟合直线")
    # 绘制观测值和拟合直线
    ax.plot(t, s, "*", label="观测值")
    plt.legend(
        loc='lower right', prop={"family": "KaiTi"}, title="图例",
        title_fontproperties={'family': 'SimHei'}
        )
    plt.tight_layout()   # 自动调整布局
    plt.savefig('fig_files/最小二乘法 Jacob（交互筛选）.png', dpi=300,bbox_inches='tight')
    plt.pause(0.01)      # 暂停更新并显示图形
    # 计算参数
    T = 0.183 * Q / beta[1]
    S = 2.25 * T / r**2 / 10**(beta[0]/beta[1])
    print(f'T = {T:.4f}, S = {S:.4e}')
    u = r**2 * S/(4 * t * T)
    print('全部数据对应的 u 值: ')
    print(u)
Q = 528 / 1440       # 抽水量 m³/min
r = 90     # 观测孔距 m
t = np.array([1, 2, 4, 6, 9, 20, 30, 40, 50, 60, 90, 120, 150,
              360, 550, 720])
s = np.array([2.5, 3.9, 6.1, 8.0, 10.6, 16.8, 20.0, 22.6, 24.7,
              26.4, 30.4, 33.0, 35.0, 42.6, 44.0, 44.5]) / 100
# 交互控件
Range_slider = widgets.IntRangeSlider(
    value=[5, 12], min=0, max=len(t)-1, step=1,
    description='选用数据:', continuous_update=False)
```

```
widgets.interact(
    fit,
    data_lims=Range_slider,
    t=widgets.fixed(t),
    s=widgets.fixed(s),
    Q=widgets.fixed(Q),
    r=widgets.fixed(r)
    );
```

选用数据: 5 – 12




```
T = 0.3172, S = 2.9447e-04
全部数据对应的 u 值:
[1.8801 0.94   0.47   0.3133 0.2089 0.094  0.0627 0.047  0.0376 0.0313
 0.0209 0.0157 0.0125 0.0052 0.0034 0.0026]
```

**练习 7**

根据 **例 2** 中观 1、观 2 孔数据，用 Jacob 公式最小二乘法求取水文地质参数。

**思考题**

1. 选择用于拟合直线的数据时，应考虑哪些问题？
2. 尝试使用观 1、观 2 全部数据进行求参，并对结果进行比较。

**提示：**

- 数组拼接可使用 `np.concatenate()` 函数（可参考 实验 2 中 Theis 公式配线法的处理方式）。
- 筛选数据时需对数组排序，`np.argsort()` 函数返回排序后的索引，按此索引提取数组元素可保持原有的对应关系不变。

# Theis 公式非线性拟合

Theis 公式为：

$$s = \frac{Q}{4\pi T}\int_u^{\infty}\frac{1}{y}e^{-y}dy = \frac{Q}{4\pi T}W(u) \tag{5.7}$$

取初始参数 $(T_0,\ S_0)$，将降深 $s$ 在 $(T_0,\ S_0)$ 处进行泰勒级数展开：

$$s(T,S) = s(T_0,S_0) + \left.\frac{\partial s}{\partial S}\right|_{S=S_0}\Delta S + \left.\frac{\partial s}{\partial T}\right|_{T=T_0}\Delta T + O(\Delta T^2 + \Delta S^2) \tag{5.8}$$

式中，$\Delta T = T - T_0,\ \Delta S = S - S_0$。由于 $\dfrac{\partial u}{\partial S} = \dfrac{u}{S},\ \dfrac{\partial u}{\partial T} = -\dfrac{u}{T}$，可得：

$$\frac{\partial s}{\partial S} = -\frac{Q}{4\pi T}\frac{e^{-u}}{S},\quad \frac{\partial s}{\partial T} = \frac{Q}{4\pi T^2}\left[-W(u) + e^{-u}\right]$$

记：

$$a = s(T_0,S_0),\quad b = \left.\frac{\partial s}{\partial S}\right|_{S=S_0},\quad c = \left.\frac{\partial s}{\partial T}\right|_{T=T_0}$$

忽略高阶无穷小项，式 (6.4) 可近似为：

$$\hat{s} = a + b\cdot\Delta S + c\cdot\Delta T \tag{5.9}$$

式中，$\hat{s}$ 为 $s$ 的预测值，$(\Delta T,\ \Delta S)$ 为搜索最优参数的步长，由 (6.6) 式可构造最小二乘问题。

以 $(\Delta T,\ \Delta S)$ 为未知系数，写成超定方程组的形式：

$$X\beta = Y \tag{5.10}$$

式中，

$$X = \begin{bmatrix} b_1 & c_1 \\ \vdots & \vdots \\ b_n & c_n \end{bmatrix},\quad \beta = \begin{bmatrix} \Delta S \\ \Delta T \end{bmatrix},\quad Y = \begin{bmatrix} s_1 - a_1 \\ \vdots \\ s_n - a_n \end{bmatrix}$$

其中，$s_i(i = 1,\cdots n)$ 为降深观测值序列，$(a_i,\ b_i,\ c_i)\ (i = 1,\cdots,n)$ 为用当前的 $(T,\ S)$ 与 $(t_i,\ r)$ 计算得出的序列。因此

$$\hat{\beta} = \begin{bmatrix} \Delta\hat{S} \\ \Delta\hat{T} \end{bmatrix} = (X^TX)^{-1}X^TY \tag{5.11}$$

由此得出一步参数优化的参数修正量。

**例 10**

根据 **例 2** 中观 2 孔数据，用 Theis 公式最小二乘法求取水文地质参数。

**Python 程序**

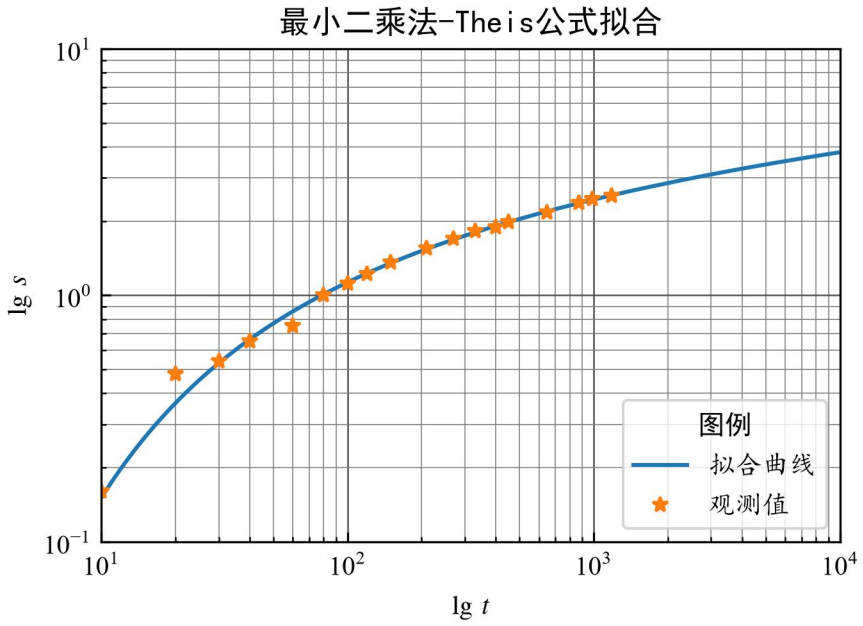
```
import numpy as np
import math
from scipy.special import exp1
import matplotlib.pyplot as plt
```

```
plt.rcParams['font.family'] = ['Times New Roman']
plt.rcParams['mathtext.fontset'] = 'stix'  # 显示数学公式（STIX 字体）
plt.rcParams['axes.unicode_minus'] = False  # 解决中文负号不能正确显示的问题
np.set_printoptions(precision=4)
def initial_parameters(t, s, Q, r):
    """基于 Cooper-Jacob 近似法估算 Theis 模型的初始参数 T 和 S"""
    idx = len(t) // 2 - 1
    slope = (s[idx+1] - s[idx]) / np.log10(t[idx+1] / t[idx])
    T = 0.183 * Q / slope
    t0 = t[idx] * 10**(-s[idx] / slope)
    S = 2.25 * T * t0 / r**2
    return T, S
def theis_drawdown(S, T, t, Q, r):
    """计算 Theis 降深 s = (Q / 4 T) * W(u), 其中 W(u) 为指数积分 E1(u)"""
    u = r**2 * S / (4 * T * t)
    return Q / (4 * np.pi * T) * exp1(u)
def theis_gradient(S, T, t, Q, r):
    """计算降深对 S 和 T 的偏导数 (ds/dS, ds/dT)"""
    u = r**2 * S / (4 * T * t)
    exp_neg_u = np.exp(-u)
    dsdS = -Q / (4 * np.pi * T * S) * exp_neg_u
    dsdT = Q / (4 * np.pi * T**2) * (-exp1(u) + exp_neg_u)
    return np.array([dsdS, dsdT]).T
def theis_residual(S, T, t, s, Q, r):
    """计算降深残差平方和 (RSS)"""
    return np.sum((s - theis_drawdown(S, T, t, Q, r))**2)
def theis_model_fit(t, s, Q, r, max_iter=1000, tol_S=1e-8, tol_T=1e-6):
    """使用高斯-牛顿法拟合 Theis 模型参数"""
    T, S = initial_parameters(t, s, Q, r)
    print(f'初始参数: S={S:.6e}, T={T:.6f}')
    iteration = 0
    for iteration in range(max_iter):
        Y = s - theis_drawdown(S, T, t, Q, r)
        X = theis_gradient(S, T, t, Q, r)
        beta = np.linalg.solve(X.T @ X, X.T @ Y)
        dS, dT = beta
        while S + dS < 0:
            dS /= 2
        while T + dT < 0:
```

```
            dT /= 2
        S += dS
        T += dT
        if abs(dS) < tol_S and abs(dT) < tol_T:
            break
    else:
        print(' 警告: 循环超过最大迭代次数, 请检查程序! ')
    return S, T, iteration
def theis_model_plot(t, s, Q, r, S, T):
    """绘制 Theis 模型拟合的双对数图"""
    # 动态计算绘图边界
    xmin = 10**math.floor(math.log10(min(t)))
    xmax = 10**math.ceil(math.log10(max(t)))
    ymin = 10**math.floor(math.log10(min(s)))
    ymax = 10**math.ceil(math.log10(max(s)))
    # 绘制双对数图
    fig, ax = plt.subplots()
    ax.set(xscale="log", yscale="log", xlim=(xmin, xmax), ylim=(ymin, ymax))
    ax.set_xlabel(r'$\lg\,t$', fontfamily="KaiTi")
    ax.set_ylabel(r'$\lg\,s$', fontfamily="KaiTi")
    ax.set_title("最小二乘法-Theis 公式拟合",fontfamily='SimHei')
    ax.grid(True, which="major", linestyle="-", linewidth=0.6, color="#505050")
    ax.grid(True, which="minor", linestyle="-", linewidth=0.4, color="#808080")
    ax.set_aspect('equal')
    # 生成平滑的拟合曲线
    x_log = np.arange(np.log10(xmin), np.log10(xmax), 0.01)
    x = np.power(10, x_log)
    y = theis_drawdown(S, T, x, Q, r)
    ax.plot(x, y, label=' 拟合曲线')
    ax.plot(t, s, '*', label=' 观测值')
    plt.legend(
        loc='lower right', prop={"family": "KaiTi"}, title="图例",
        title_fontproperties={'family': 'SimHei'}
        )
    plt.tight_layout()   # 自动调整布局
    plt.savefig('fig_files/最小二乘法 Theis.png',dpi=300,bbox_inches='tight')
    plt.show()      # 暂停更新并显示图形
# 原始数据
def single_data():
```

```
    r, Q = 140.0, 1440  # m, m³/d
    t = np.array([10, 20, 30, 40, 60, 80, 100, 120, 150, 210, 270,
                  330, 400, 450, 645, 870, 990, 1185])
    # 单位天转分钟
    Q = Q / 1440.0
    s = np.array([0.16, 0.48, 0.54, 0.65, 0.75, 1.0, 1.12, 1.22,
                  1.36, 1.55, 1.7, 1.83, 1.89, 1.98, 2.17, 2.38,
                  2.46, 2.54])
    return t, s, Q, r
# 主程序
if __name__ == "__main__":
    # 1. 获取数据
    t, s, Q, r = single_data()
    # 2. 拟合模型
    S, T, iterations = theis_model_fit(t, s, Q, r)
    # 3. 绘制拟合结果
    theis_model_plot(t, s, Q, r, S, T)
    # 4. 输出最终结果
    print(f' 循环次数 = {iterations}')
    print(f'T = {T:.2f} m²/d, S = {S:.4e}')
    print(f'RSS = {theis_residual(S, T, t, s, Q, r):.4e}')
```

初始参数: S=2.180116e-04, T=0.140744




循环次数 = 3

T = 0.13 m²/d, S = 2.5012e-04

```
RSS = 2.8709e-02
```

**注意：**

- 最小二乘法得出的参数修正量可能导致参数为负值，需采用步长折半等策略确保参数的合理性。
- 初值对结果有重要影响，建议采用 Jacob 公式计算合理的初始值。
- 可进行多次迭代直至满足收敛条件。

# 实验 6 一维显式有限差分法

## 有限差分基础知识

有限差分法是数值求解微分方程的经典方法。其基本思想是将求解区域划分为规则网格，将微分方程中的导数（偏导数）用网格节点上的函数差商进行近似，从而将微分方程转化为线性代数方程组，最终计算出各节点上的函数值。

设光滑函数 $f(x)$，记步长 $\Delta x = h$，其泰勒展开式为：

$$f(x+h) = f(x) + hf'(x) + \frac{h^2}{2!}f''(x) + \frac{h^3}{3!}f'''(x) + \cdots \tag{6.1}$$

用差商近似表示微分时，被截去的高阶项所产生的误差称为**截断误差**，亦称**离散误差**。

**一阶向前差分**

$$f'(x) \approx \frac{f(x+h) - f(x)}{h} \tag{6.2}$$

截断误差为 $O(h)$，具有**一阶精度**。

**一阶向后差分**

$$f'(x) \approx \frac{f(x) - f(x-h)}{h} \tag{6.3}$$

截断误差为 $O(h)$，具有**一阶精度**。

**一阶中心差分**

将 $f(x+h)$ 与 $f(x-h)$ 的泰勒展开式相减，可得：

$$f'(x) \approx \frac{f(x+h) - f(x-h)}{2h} \tag{6.4}$$

截断误差为 $O(h^2)$，具有**二阶精度**。

**二阶中心差分**

将 $f(x+h)$ 与 $f(x-h)$ 的泰勒展开式相加，可得：

$$f"(x) \approx \frac{f(x+h) - 2f(x) + f(x-h)}{h^2} \tag{6.5}$$

截断误差为 $O(h^2)$，具有**二阶精度**。

## 显示差分与隐式差分格式

以一维定解问题（**Dirichlet 边界条件**）为例：

$$\begin{cases} a\dfrac{\partial^2 u}{\partial x^2} = \dfrac{\partial u}{\partial t} & (0 < x < l, t > 0) \\ u(x,0) = u_0(x) & (0 \le x \le l) \\ u(0,t) = \varphi_1(t) & (t > 0) \\ u(l,t) = \varphi_2(t) & (t > 0) \end{cases} \tag{6.6}$$

设空间步长 $h = \dfrac{l}{m}$，时间步长为 $\tau$，节点坐标为 $x_j = jh$ $(j = 0, 1, \dots, m)$，时间层为 $t_n = n\tau$ $(n = 0, 1, 2, \dots)$，记 $u_j^n = u(x_j, t_n)$。定义**网格比** $r = \dfrac{a\tau}{h^2}$。

## 显式差分格式

对节点 $(x_j, t_n)$，时间导数采用**向前差分**，空间二阶导数采用**二阶中心差分**，可得：

$$\frac{u_j^{n+1} - u_j^n}{\tau} = a^2 \cdot \frac{u_{j+1}^n - 2u_j^n + u_{j-1}^n}{h^2} \tag{6.7}$$

其截断误差为 $O(\tau + h^2)$，具有**一阶时间精度与二阶空间精度**。

整理上式，得到递推公式：

$$u_j^{n+1} = r\,u_{j-1}^n + (1 - 2r)\,u_j^n + r\,u_{j+1}^n \quad (j = 1, 2, \dots, m-1,\ n \ge 0) \tag{6.8}$$

利用定解条件：

$$\begin{cases} u_j^0 = u_0(x_j), & j = 0, 1, \dots, m \\ u_0^n = \varphi_1(t_n), \quad u_m^n = \varphi_2(t_n), & n \ge 1 \end{cases} \tag{6.9}$$

即可逐层计算 $u_j^{n+1}$。

对于每一时间层，显式差分格式无需求解方程组，但为获得稳定解，需满足稳定性条件 $r \le 0.5$。

显式差分格式的误差主要由空间步长主导，时间步长影响较小。可先设定 $r(\le 0.5)$ 与 $h$，再确定 $\tau$；若误差仍不满足要求，可通过加密网格来平衡计算速度与精度。

## 隐式差分格式

对节点 $(x_j, t_n)$，时间导数采用**向后差分**，空间二阶导数取在 $t_{n+1}$ 层：

$$\frac{u_j^{n+1} - u_j^n}{\tau} = a^2 \cdot \frac{u_{j+1}^{n+1} - 2u_j^{n+1} + u_{j-1}^{n+1}}{h^2} \tag{6.10}$$

整理得：

$$-r\,u_{j-1}^{n+1} + (1 + 2r)\,u_j^{n+1} - r\,u_{j+1}^{n+1} = u_j^n \quad (j = 1, 2, \dots, m-1,\ n \ge 0) \tag{6.11}$$

定解条件为：

$$\begin{cases} u_j^0 = u_0(x_j), & j = 0, 1, \dots, m \\ u_0^{n+1} = \varphi_1(t_{n+1}), \quad u_m^{n+1} = \varphi_2(t_{n+1}), & n \ge 0 \end{cases} \tag{6.12}$$

隐式差分格式的截断误差与显式格式相同，均为 $O(\tau + h^2)$，具有**一阶时间精度与二阶空间精度**。

对于每一时间层，隐式差分格式均需求解一个系数矩阵为三对角阵的线性方程组，但该格式是**无条件稳定**的。

## Neumann 边界条件

边界推荐为 **Dirichlet** 类型时，边界节点上的 $u$ 值已知，可直接代入计算；若边界条件为 **Neumann** 类型时，边界节点的 $u$ 值未知，需进行特别处理。

**虚拟节点法（Ghost Point Method）**

**左边界：**

$$\left.\frac{\partial u}{\partial x}\right|_{x=x_0} = \psi_1 \tag{6.13}$$

在左侧引入虚拟节点 $x_{-1}$，将边界条件用中心差分格式表示：

$$\psi_1 = \frac{u_1 - u_{-1}}{2h} \implies u_{-1} = u_1 - 2h\,\psi_1 \tag{6.14}$$

二阶偏导数同样采用中心差分格式：

$$\left.\frac{\partial^2 u}{\partial x^2}\right|_{x=x_0} = \frac{u_1 - 2u_0 + u_{-1}}{h^2} \tag{6.15}$$

将 $u_{-1}$ 表达式代入并化简：

$$\left.\frac{\partial^2 u}{\partial x^2}\right|_{x=x_0} = \frac{2\,(u_1 - u_0 - h\,\psi_1)}{h^2} \tag{6.16}$$

- **显式差分格式**

$$\frac{u_0^{n+1} - u_0^n}{\tau} = \frac{2a\,(u_1^n - u_0^n - h\,\psi_1)}{h^2} \tag{6.17}$$

整理得：

$$u_0^{n+1} = u_0^n + 2r\,(u_1^n - u_0^n - h\,\psi_1) \tag{6.18}$$

- **隐式差分格式**

$$\frac{u_0^{n+1} - u_0^n}{\tau} = \frac{2a\left(u_1^{n+1} - u_0^{n+1} - h\,\psi_1\right)}{h^2} \tag{6.19}$$

整理得：

$$(1 + 2r)\,u_0^{n+1} - 2r\,u_1^{n+1} = u_0^n - 2rh\,\psi_1 \tag{6.20}$$

**右边界**:

$$\left.\frac{\partial u}{\partial x}\right|_{x=x_m} = \psi_2 \tag{6.21}$$

在右侧引入虚拟节点 $x_{m+1}$，同样用中心差分格式表示边界条件：

$$\psi_2 = \frac{u_{m+1} - u_{m-1}}{2h} \implies u_{m+1} = u_{m-1} + 2h\,\psi_2 \tag{6.22}$$

二阶偏导数用中心差分格式表示：

$$\left.\frac{\partial^2 u}{\partial x^2}\right|_{x=x_m} = \frac{u_{m+1} - 2u_m + u_{m-1}}{h^2} \tag{6.23}$$

将 $u_{m+1}$ 表达式代入并化简：

$$\left.\frac{\partial^2 u}{\partial x^2}\right|_{x=x_m} = \frac{2\left(u_{m-1} - u_m + h\,\psi_2\right)}{h^2} \tag{6.24}$$

- **显式差分格式**

$$\frac{u_m^{n+1} - u_m^n}{\tau} = \frac{2\left(u_{m-1}^n - u_m^n + h\,\psi_2\right)}{h^2} \tag{6.25}$$

整理得：

$$u_m^{n+1} = u_m^n + 2r\left(u_{m-1}^n - u_m^n + h\,\psi_2\right) \tag{6.26}$$

- **隐式差分格式**

$$\frac{u_m^{n+1} - u_m^n}{\tau} = \frac{2\left(u_{m-1}^{n+1} - u_m^{n+1} + h\,\psi_2\right)}{h^2} \tag{6.27}$$

整理得：

$$-2r\,u_{m-1}^{n+1} + (1+2r)\,u_m^{n+1} = u_m^n + 2rh\,\psi_2 \tag{6.28}$$

**例 11**

考虑如下定解问题：

$$\begin{cases} \dfrac{\partial^2 u}{\partial x^2} = \dfrac{\partial u}{\partial t} \\ u(x,0) = \sin(\frac{\pi x}{2}) \\ u\big|_{x=0} = 0 \qquad\qquad t > 0 \\ \frac{\partial u}{\partial x}\big|_{x=1} = 0 \qquad\qquad t > 0 \end{cases} \tag{6.29}$$

其精确解为：

$$u(x,t) = \exp(-\frac{\pi^2 t}{4})\sin\left(\frac{\pi x}{2}\right) \tag{6.30}$$

用显式差分格式求 t = 0.1, 0.5, 1.0 时的解。在下面的主程序中，改变 `t_calc` 即可。

**Python 代码**

```python
import numpy as np
import matplotlib.pyplot as plt
plt.rcParams['font.family'] = ['Times New Roman']
plt.rcParams['mathtext.fontset'] = 'stix'  # 显示数学公式（STIX 字体）
plt.rcParams['axes.unicode_minus'] = False  # 解决中文负号不能正确显示的问题
np.set_printoptions(precision=4)
def exact_sol(x, t):
    """
    方程解析解
    """
```

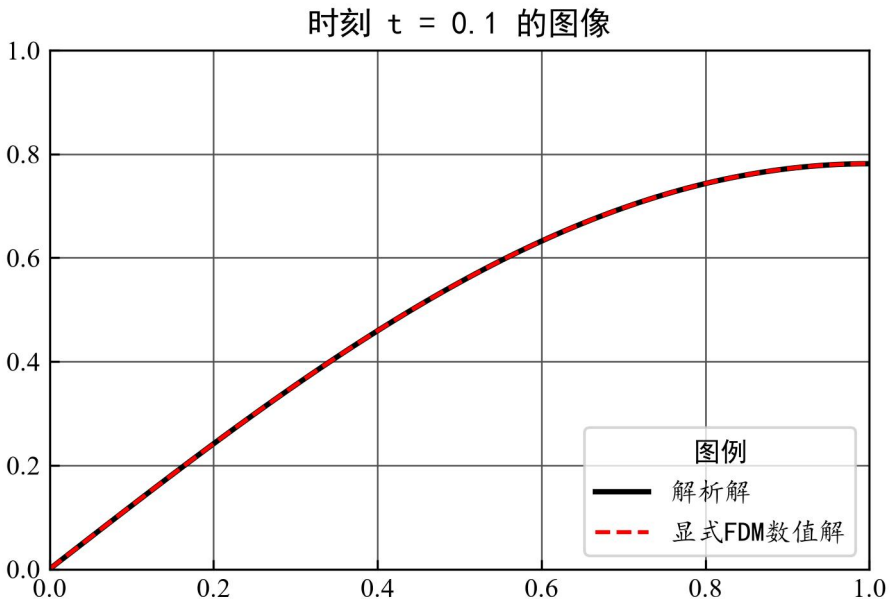
```
    return np.exp(-np.pi**2 / 4 * t) * np.sin(np.pi * x / 2)
def explicit_fdm(init_func, a, t_end, nx=101, dt_safety=0.5,
                 bc_left_type='dirichlet', bc_right_type='dirichlet',
                 bc_left_val=0, bc_right_val=0):
    """
    显式有限差分法 a·u_xx = u_t
    Neumann 边界采用虚拟节点二阶格式，支持非零导数值
    """
    dx = 1.0 / (nx - 1)
    dt = dx ** 2 * dt_safety
    nt = int(t_end / dt)
    x = np.linspace(0, 1, nx)
    u = init_func(x)
    for _ in range(nt):
        u_prev = u.copy()
        # 内部节点：二阶中心差分格式
        u[1:-1] = u_prev[1:-1] + a * dt * (
            u_prev[2:] - 2 * u_prev[1:-1] + u_prev[:-2]) / dx**2
        # 左边界
        if bc_left_type == 'dirichlet':
            u[0] = bc_left_val
        elif bc_left_type == 'neumann':
            g = bc_left_val
            # 构造左侧虚拟节点：u_{-1} = u_1 - 2*g*dx
            u_left_vir = u_prev[1] - 2 * g * dx
            u[0] = u_prev[0] + a * dt * (
                u_left_vir - 2 * u_prev[0] + u_prev[1]) / dx**2
        # 右边界
        if bc_right_type == 'dirichlet':
            u[-1] = bc_right_val
        elif bc_right_type == 'neumann':
            g = bc_right_val
            # 构造右侧虚拟节点：u_{N+1} = u_{N-1} + 2*g*dx
            u_right_vir = u_prev[-2] + 2 * g * dx
            u[-1] = u_prev[-1] + a * dt * (
                u_prev[-2] - 2 * u_prev[-1] + u_right_vir) / dx**2
    return x, u, dt, nt
def run_compare(u_exact, init_func, a, t_test=0.1, dt_safety=0.45,
                bc_left_type='dirichlet', bc_right_type='dirichlet',
```

```
                bc_left_val=0, bc_right_val=0, plot_exact=False):
    """计算数值解，与解析解对比、计算误差并绘图"""
    print("=" * 70)
    print("Neumann 边界：虚拟节点二阶格式")
    print("=" * 70)
    x, u_num, dt, nt = explicit_fdm(
        init_func, a, t_test, nx=101, dt_safety=dt_safety,
        bc_left_type=bc_left_type, bc_right_type=bc_right_type,
        bc_left_val=bc_left_val, bc_right_val=bc_right_val
    )
    print(f"时间步长 = {dt:.2e} | 迭代步数 = {nt}")
    # 只有当传入了解析解，且 plot_exact 为 True 时，才计算误差
    u_ana = None
    max_err = None
    if plot_exact:
        u_ana = u_exact(x, t_test)
        max_err = np.max(np.abs(u_num - u_ana))
        print(f"最大误差 = {max_err:.2e}")
    # 绘制对比曲线
    fig, ax = plt.subplots()
    # 只有当传入了解析解，且 plot_exact 为 True 时，才绘制解析解
    if plot_exact:
        ax.plot(x, u_ana, 'k-', linewidth=2, label='解析解')
    ax.plot(x, u_num, 'r--', linewidth=1.5, label='显式 FDM 数值解')
    ax.set(xlim=(0, 1), ylim=(0, 1))
    ax.grid(True, which="major", linestyle="-", linewidth=0.6, color="#505050")
    ax.set_title(
        f'时刻 t = {t_test} 的图像', fontfamily='SimHei'
        )
    ax.legend(
        loc='lower right', prop={"family": "KaiTi"}, title="图例",
        title_fontproperties={'family': 'SimHei'}
        )
    plt.show()
    return fig, ax

if __name__ == "__main__":
    # 模型参数设置
    coeff_a = 1.0
```

```
    init_cond = lambda x: np.sin(np.pi * x / 2)
    t_calc = 0.1
    # 边界条件设置
    left_bc_type  = 'dirichlet'
    left_bc_val   = 0.0
    right_bc_type = 'neumann'
    right_bc_val  = 0.0
    # 场景 1: 保留解析解功能, 但不绘出解析解
    run_compare(
        exact_sol,
        init_func=init_cond,
        a=coeff_a,
        t_test=t_calc,
        dt_safety=0.45,
        bc_left_type=left_bc_type,
        bc_right_type=right_bc_type,
        bc_left_val=left_bc_val,
        bc_right_val=right_bc_val,
        plot_exact=True
    )
```

```
==========================================================================
Neumann 边界: 虚拟节点二阶格式
==========================================================================
时间步长 = 4.50e-05 | 迭代步数 = 2222
最大误差 = 1.25e-05
```


时刻 t = 0.1 的图像
1.0
0.8
0.6
0.4
0.2
0.0
0.0
0.2
0.4
0.6
0.8
1.0
图例
解析解
显式FDM数值解

# 附录井函数计算

## Theis 井函数

$$W(u)=\int_u^{\infty}\frac{\mathrm{e}^{-y}}{y}\mathrm{d}y=\mathrm{E}_1(u)=-\mathrm{E}_i(-u) \tag{6.31}$$

`scipy.special.exp1` 函数可直接计算该井函数，亦可采用多项式逼近。

**$W(u)$ 的多项式逼近**

- $0<u\leq 1$ 时

$$W(u)=-\ln u+a_0+a_1u+a_2u^2+a_3u^3+a_4u^4+a_5u^5 \tag{6.32}$$

式中

$$\begin{aligned} a_0&=-0.57721566 & a_3&=0.05519968\\ a_1&=0.99999193 & a_4&=-0.00976004\\ a_2&=-0.24991055 & a_5&=0.00107857 \end{aligned} \tag{6.33}$$

- $1<u<\infty$ 时

$$W(u)=\frac{b_0+b_1u+b_2u^2+b_3u^3+u^4}{c_0+c_1u+c_2u^2+c_3u^3+u^4}\cdot\frac{e^{-u}}{u} \tag{6.34}$$

式中

$$\begin{aligned} b_0&=0.2677737343 & c_0&=3.9584969228\\ b_1&=8.6347608925 & c_1&=21.0996530827\\ b_2&=18.0590169730 & c_2&=25.6329561486\\ b_3&=8.5733287401 & c_3&=9.5733223454 \end{aligned} \tag{6.35}$$

**多项式逼近代码**

```python
import numpy as np
import scipy.special as sps
np.set_printoptions(precision=4)

def theis_poly(u):
    """多项式逼近计算 Theis 井函数，u 可为数组"""
    a = [-0.57721566, 0.99999193, -0.24991055,
         0.05519968, -0.00976004, 0.00107857]
```

```
    b = [0.2677737343, 8.6347608925, 18.059016973, 8.5733287401]
    c = [3.9584969228, 21.0996530827, 25.6329561486, 9.5733223454]
    def w(u):
        if u <= 1:
            return -np.log(u) + a[0] + u*(a[1] + u*(a[2] + u*
            (a[3] + u*(a[4] + u*a[5]))))
        else:
            num = b[0] + u*(b[1] + u*(b[2] + u*(b[3] + u)))
            den = c[0] + u*(c[1] + u*(c[2] + u*(c[3] + u)))
            return num/den * np.exp(-u)/u
    return np.vectorize(w)(u)
# 测试
u_test = np.logspace(-4, 0, 5)
print("   u:", u_test)
print("exp1:", sps.exp1(u_test))
print("poly:", theis_poly(u_test))
```

```
   u: [1.e-04 1.e-03 1.e-02 1.e-01 1.e+00]
exp1: [8.6332 6.3315 4.0379 1.8229 0.2194]
poly: [8.6332 6.3315 4.0379 1.8229 0.2194]
```

## Hantush-Jacob 井函数

$$W(u,\beta)=\int_u^\infty \frac{1}{y}\exp\Big(-y-\frac{\beta^2}{4y}\Big)dy,\quad W(u,\beta)=2K_0(\beta)-W\left(\frac{\beta^2}{4u},\beta\right) \tag{6.36}$$

式中：$u=\frac{r^2S}{4Tt}$，$\beta=\frac{r}{B}$.

**级数形式**（Hunt，1977）

$$W(u,\beta)=\sum_{n=0}^{\infty}\left(-\frac{\beta^2}{4u}\right)^n\frac{E_{n+1}(u)}{n!} \tag{6.37}$$

式中，

$$E_n(u)=\int_1^\infty \frac{e^{-uy}}{y^n}dy=u^{n-1}\int_u^\infty \frac{e^{-y}}{y^n}dy\quad (n=0,1,2,\cdots;\mathrm{Re}\ u>0) \tag{6.38}$$

为指数积分。当 $\dfrac{\beta^2}{4u}<1$ 时，该级数快速收敛。

`scipy.special.expn` 函数可计算指数积分，`scipy.special.k0` 函数可计算 0 阶第二类修正 Bessel 函数 $K_0(x)$。

具体实现可参见实验 2 中的 `hantush_jacob` 函数。